SESSION 2 · INSTRUCTOR ANSWER KEY

Problem Set

Sixty-eight worked problems across fourteen topics

Everything here is hand-workable. No calculator, no notebook, no library. Basic problems use clean numbers; the advanced ones use realistic figures that still resolve on paper.

Each problem is followed immediately by a full narrative solution — the reasoning, not just the steps — and a note on the mistake students actually make.

Problems are graded by role, and only the roles that genuinely need a topic appear in it. There is no honest GenAI-engineer problem about permutations, and no honest analyst problem about attention shapes.

Where a topic ends in a judgement call rather than a number, the last problem has no arithmetic in it at all.

THE THREE ROLE BANDS

A · Analyst

Reads outputs and decides what they mean. Needs the arithmetic to be correct and the interpretation to be honest. Rarely touches a model.

M · ML engineer

Builds and trains. Needs shapes, gradients, thresholds and evaluation to be second nature. Debugs at three in the morning.

G · GenAI engineer

Assembles systems from models. Needs retrieval, sampling, serving cost and agent behaviour. Owns the failure modes nobody warned them about.

ON THE NUMBER OF PROBLEMS

Every topic opens with a short note on how many problems earn competence in it, and why that number. The pattern across all fourteen is the same, so it is worth stating once here.

Below three problems, nothing sticks. One problem teaches the procedure. The second reveals which part of the first was accidental. The third is where a student stops re-reading the worked example and starts reaching for the method directly. Two problems produce recognition without recall; students will nod at your solution and be unable to start their own.

Above eight, people stop finishing. Not because it is too hard — because the marginal problem teaches nothing new, and a set that is visibly unfinishable gets abandoned entirely rather than partially. Six well-chosen problems get completed; twelve get skimmed.

The right number depends on how many distinct traps a topic has, not on how important it is. Counting has exactly two traps — order versus no order, and pool versus one-from-each-list — so five problems is generous. Probability has five separate traps, which is why it gets six problems and could justify eight. Numerical stability has one idea and one trap, so three problems is the honest ceiling; a fourth would be padding.

TOPIC 01 · FIVE PROBLEMS

Counting & combinations

Two traps only: does order matter, and is this one pool or several lists. Five problems is enough to make both reflexive.

How many problems earn competence here: five. Two to separate permutations from combinations, one to expose the pool-versus-lists distinction, one on multiplication of independent choices, and one judgement problem. A sixth would be a fourth variation on "does order matter", which nobody needs.

1.1 Shortlisting for a kirana loyalty pilot

A

Problem. A retail chain wants to pilot a loyalty programme in 4 of its 9 city stores. Head office will treat all four pilot stores identically — same signage, same offer, same launch date. How many different sets of four stores could they choose?

Solution. Start where you should always start, which is not the formula. Ask whether swapping two of your picks changes the answer.

Suppose the chosen stores are Andheri, Bandra, Colaba and Dadar. Now list them in a different order — Dadar, Colaba, Bandra, Andheri. Has anything changed about the pilot? No. The same four shops get the same signage on the same morning. There is no "first" store, because head office is treating them identically. Order does not matter, so this is a combination.

Now build the answer rather than substituting into a formula. Begin by over-counting deliberately: how many ways can all nine stores be lined up in a queue? Nine factorial, which is 362,880.

That number is answering a much bigger question than the one asked, so we now remove the two kinds of over-counting. First, the four stores we selected can be arranged among themselves in 4! = 24 ways, and all 24 describe the identical pilot. Divide by 24. Second, the five stores we did not select can be shuffled in 5! = 120 ways, and nobody cares in what order they were passed over. Divide by 120.

So: 362,880 ÷ 24 ÷ 120. Take it in two steps — 362,880 ÷ 24 = 15,120, and 15,120 ÷ 120 = 126.

Answer: 126 possible sets of four stores.

Worth a sanity check, since it costs nothing. Use the symmetry: choosing 4 to include is the same act as choosing 5 to exclude, so 9C4 must equal 9C5. And there is a faster route to the same number — 9 × 8 × 7 × 6 ÷ 24, which is 3,024 ÷ 24 = 126. Same answer, arithmetic you can do in your head. Teach students to cancel the factorials before multiplying them out; 362,880 is a number nobody needs to write down.

Common mistake. Computing 9P4 = 3,024 and stopping. The student has correctly worked out how many ordered shortlists exist and has forgotten to divide by 4!. Ask them which four stores are "first" in a pilot where everything launches on the same day — the question answers itself, and the missing division becomes obvious.

1.2 The same nine stores, ranked

A

Problem. Head office changes its mind. Instead of four identical pilots, they will run a staged rollout: one store in January, one in February, one in March, one in April — still four stores out of nine. How many rollout plans are possible now?

Solution. Nothing about the stores changed. Only the meaning of the selection changed, and that is the entire point of running these two problems back to back.

Ask the same question. Andheri in January, Bandra in February, Colaba in March, Dadar in April. Now swap Andheri and Dadar. Is that the same plan? Absolutely not — a different store now opens first, three months earlier, with different cashflow and different learnings feeding into the rest. Order matters. This is a permutation.

So we still over-count from 9!, but we now remove only *one* kind of duplicate rather than two. The five stores left out can still be shuffled meaninglessly, so we divide by 5! = 120. But the four selected stores must *not* be divided out, because their ordering is exactly what we are counting.

362,880 ÷ 120 = 3,024. Or directly, which is easier: 9 × 8 × 7 × 6 = 3,024 — nine choices for January, then eight remaining for February, seven for March, six for April.

Answer: 3,024 rollout plans.

Put the two numbers side by side and make the class say the relationship out loud: 3,024 ÷ 126 = 24, which is 4!. Every one of the 126 store-sets can be sequenced in 24 different orders. Permutations equal combinations times orderings. If a student can reconstruct that sentence, they never need to memorise either formula again — one gives the other.

Common mistake. Reusing the previous answer of 126 because "it's the same nine stores and the same four picks". The student is anchoring on the objects rather than on the question. The stores are identical; the *decision* is different. This is why these two problems must be taught adjacently.

1.3 Sizing a training sweep before promising a date

M

Problem. You plan to tune a churn model. You will try 5 learning rates, 4 batch sizes, 3 dropout values and 2 optimizers, in every combination. Each run takes 35 minutes on your single GPU. Your manager asks whether you can have results by Friday — it is Monday morning. What do you say?

Solution. First, identify the structure, because this is the problem that separates the two counting patterns.

A student who has just done problems 1.1 and 1.2 will reach for a combination formula. Stop them. Ask: am I taking several items out of one pool, or exactly one item from each of several separate lists?

Here, the learning rates and the batch sizes are different lists entirely. You are not choosing four settings from a bag of fourteen. You must pick exactly one learning rate *and* exactly one batch size *and* one dropout *and* one optimizer. Every combination is legal, and no choice uses up another.

When it is one-from-each-list, the counts multiply. Why multiply rather than add? Because for each of the 5 learning rates you must try all 4 batch sizes — that is 5 groups of 4, which is 20. Then for each of those 20 pairs, all 3 dropout values: 20 groups of 3, so 60. Then both optimizers: 60 groups of 2, so 120.

5 × 4 × 3 × 2 = **120 runs.**

Now convert to time, because the count alone does not answer the manager's question. 120 runs × 35 minutes = 4,200 minutes. Divide by 60: 70 hours. Divide by 24: 2.9 days of continuous GPU time.

Answer: 120 runs, 70 hours, so roughly three days of unbroken compute. Monday morning to Friday is four days, so yes — but only if nothing fails, nothing else needs that GPU, and you do not want to iterate on the results. The honest answer is "yes for the sweep, no for the sweep plus a second pass", and that second sentence is the one worth saying out loud.

Then show them the fragility. Someone suggests adding just two weight-decay values — a small, reasonable request. That is not plus two runs. It is × 2: 240 runs, 140 hours, 5.8 days. One innocuous suggestion in a corridor destroyed the deadline, and nobody in that conversation did any arithmetic.

Common mistake. Adding instead of multiplying — 5 + 4 + 3 + 2 = 14 runs, so "about eight hours, easy". This is the single most expensive counting error in practice, because the answer is small, plausible, and gets promised to somebody. Have the student draw the tree for just the first two lists: five branches, each splitting into four. Twenty leaves, not nine.

1.4 Prompt-variant testing for a support assistant

G

Problem. You are evaluating a RAG assistant for a bank's support desk. You want to test 3 system prompts, 4 chunk sizes, 2 embedding models, and 3 values of k (how many chunks to retrieve). Each configuration must be run against an evaluation set of 200 questions, and each question costs about ₹0.40 in API calls. What is the total number of configurations, the total number of model calls, and the cost? Then: what would you actually do?

Solution. Same structure as 1.3 — one choice from each of four separate lists — so the configuration count multiplies.

3 × 4 × 2 × 3. Take it in pairs: 3 × 4 = 12, and 2 × 3 = 6, then 12 × 6 = **72 configurations.**

Now the calls. Each configuration runs the full evaluation set, so 72 × 200 = **14,400 model calls.**

And the cost: 14,400 × ₹0.40 = **₹5,760.**

Pause on that number, because the interesting thing about it is that it is *small*. Under six thousand rupees to evaluate seventy-two system configurations properly. Which means the constraint here is not money at all — and identifying which resource actually binds is most of the value of doing this arithmetic.

So what does bind? Two things. Time, if the calls are sequential and rate-limited: at, say, two seconds a call, 14,400 calls is eight hours — one working day, which is fine. And *your* time, if any configuration needs a human to read outputs, because 72 × 200 responses is 14,400 pieces of text and nobody is reading those.

What you would actually do: not the full grid. Notice the four knobs are not equally important, and — critically — they are not equally entangled. The embedding model and chunk size interact strongly, so test those two together as a 4 × 2 = 8-configuration grid with everything else fixed. Pick the winner. Then sweep k over 3 values, and finally the 3 prompts. That is 8 + 3 + 3 = 14 configurations instead of 72, and it finds nearly the same answer.

That is the real lesson of this topic for a GenAI engineer: counting the grid is what tells you the full grid was never necessary. Not that it was unaffordable — here it was perfectly affordable — but that 58 of the 72 runs would have told you nothing you did not already know.

Common mistake. Forgetting to multiply by the evaluation set size. Students report "72 runs, that's cheap" and lose the factor of 200 entirely — which is the factor that turns a trivial number into a real one. Any time a configuration is evaluated against a dataset, the dataset size is part of the cost, and it is the part people omit.

1.5 A ranking claim in a design review

M · G

Problem. No arithmetic in this one. In a design review, an engineer proposes: "for each search query, we'll generate every possible ordering of the top 12 results, score each ordering with our ranking model, and serve the best one." What do you say, and what should the system do instead?

Solution. What you say is: that is 12! orderings, which is 479 million, per query. And you should be able to reach that number without a calculator by knowing the landmarks — 10! is about 3.6 million, so 12! is that times 11 times 12, roughly 480 million. At even a microsecond per scored ordering, that is eight minutes of compute for one search. The proposal is not slow; it is impossible by about nine orders of magnitude.

But the point of this problem is not the number. It is that the number forces a specific architecture, and that architecture is the one every search system in the world actually uses.

What the system should do instead: score each result independently — one number per result, 12 scores rather than 479 million — and then sort. Sorting 12 items is trivial. You have replaced "evaluate every arrangement" with "evaluate every item, then arrange".

And it is worth being honest with the class about what that substitution costs, because it is not free. Independent scoring cannot express relationships between results. It cannot say "these two results are near-duplicates, so demote the second" or "this list needs one recent item near the top for diversity". Those are properties of the *arrangement*, and we just gave up the ability to evaluate arrangements.

Which is exactly why real ranking systems have a second stage: score-and-sort to get a candidate list, then a small re-ranker that considers a handful of orderings — dozens, not hundreds of millions. The counting fact did not just rule out an approach. It dictated a two-stage architecture, and now the class knows why every serious retrieval system has one.

Common mistake. Answering only "that's too slow, use score-and-sort" and stopping. Correct, and half an answer. The valuable half is naming what score-and-sort *cannot* do — because a student who does not know that will later be baffled when their search results are three near-identical documents. Push for the trade-off, not just the fix.

TOPIC 02 · SIX PROBLEMS

Probability & Bayes

The heaviest set in the document. Five distinct traps, each of which ruins a real decision when missed.

How many problems earn competence here: six, and eight would be defensible. The traps are: confusing P(A|B) with P(B|A); ignoring the base rate; adding probabilities that should be multiplied; forgetting that the denominator must include both routes to the evidence; and treating a posterior as a decision. One problem per trap, plus one that combines them. This is the topic where a thin problem set does the most damage, because every error it permits is invisible — the arithmetic looks fine and the conclusion is wrong.

2.1 Two conditions on a support queue

A

Problem. A support desk logged 400 tickets last month. 120 were about billing. 80 were escalated to a manager. 30 tickets were both about billing and escalated. (a) What is the probability a random ticket is about billing? (b) Given that a ticket was escalated, what is the probability it is about billing? (c) Given that a ticket is about billing, what is the probability it was escalated?

Solution. Three questions, one dataset, and three different denominators. That is the whole lesson.

(a) Plain probability. Favourable outcomes over all outcomes: 120 billing tickets out of 400 total. 120 ÷ 400 = 0.30. Thirty percent.

Notice the denominator: 400, every ticket that exists. Nothing has been told to us, so the whole world is still in play.

(b) Now we are told the ticket was escalated. And here is the move that matters: that information does not change any number in the dataset. Nobody re-labelled a ticket. What changed is which tickets are still candidates.

Only 80 tickets were escalated. The other 320 are off the table — they cannot be our ticket, because our ticket was escalated. So 80 is the new denominator; the escalated tickets are now the entire world.

Within that world of 80, how many are about billing? The 30 that are both. So P(billing | escalated) = 30 ÷ 80 = **0.375.**

(c) Now flip the condition. We are told the ticket is about billing, so the world shrinks to the 120 billing tickets. Of those, how many were escalated? Again the 30 that are both — the same numerator as before.

P(escalated | billing) = 30 ÷ 120 = **0.25.**

Put (b) and (c) together and make the class read them aloud: 0.375 and 0.25. Same two events. Same 30 tickets on top. Completely different numbers, because the denominators are different — 80 versus 120.

That is the mechanical explanation of why P(A|B) ≠ P(B|A), and it is worth having students see it as a division rather than as an abstract warning. The numerator of a conditional probability is always the overlap. Only the denominator moves, and the denominator is whatever you were told.

Common mistake. Using 400 as the denominator in (b) and (c) — giving 30 ÷ 400 = 0.075 for both, which at least has the virtue of being obviously wrong twice. The student has computed P(billing AND escalated), which is a real quantity and is not what was asked. Ask them: "if I have already told you the ticket was escalated, why are the 320 non-escalated tickets still in your denominator?"

2.2 A UPI fraud alert, counted in people

A · M

Problem. A bank's fraud model reviews UPI transactions. Genuinely fraudulent transactions are 0.5% of the total. The model flags 92% of real frauds. It also flags 4% of legitimate transactions. A transaction has just been flagged. What is the probability it is actually fraudulent?

Solution. Do not use fractions. Use transactions. Percentages of percentages are genuinely hard to hold in the head, and whole objects in a pile are easy — this single technique is why this problem is workable on paper.

Take 100,000 transactions. Choose a round number large enough that no group rounds to a fraction.

Step 1 — split by reality, before the model says anything. 0.5% of 100,000 is 500 fraudulent. The remaining 99,500 are legitimate.

Stop here and look at those two numbers, because the answer is already visible to anyone who knows what to look for. The legitimate group is 199 times larger. A model can be very accurate on the small group and still be swamped by its errors on the large one.

Step 2 — true positives. Of the 500 frauds, the model catches 92%. 0.92 × 500 = 460. It missed 40, which we will need later for recall but not for this question.

Step 3 — false positives. Of the 99,500 legitimate transactions, 4% get flagged anyway. 4% of 99,500: one percent is 995, so four percent is 3,980.

Now compare. 460 real catches. 3,980 false alarms. A 4% error rate on a huge group produced almost nine times more alerts than a 92% success rate on a small one.

Step 4 — total flagged. 460 + 3,980 = 4,440 transactions carry a flag.

Step 5 — the answer. Our transaction is one of those 4,440, and that is genuinely all we know about it. Of the 4,440, how many are fraudulent? 460.

460 ÷ 4,440 = **0.104, about 10.4%.**

So a flagged transaction is about 90% likely to be perfectly legitimate — from a model that catches 92% of fraud. Both facts are true simultaneously, and neither is a criticism of the model.

Connect it back to the formula explicitly, or students will think they have learned a trick rather than Bayes. The 4,440 is P(flagged) — the evidence. The 460 is P(flagged | fraud) × P(fraud), likelihood times prior. And 460 ÷ 4,440 is posterior = likelihood × prior ÷ evidence. We did not avoid Bayes' theorem; we evaluated it in units of transactions instead of decimals.

Common mistake. Answering 92%. The student has reported P(flagged | fraud) when asked for P(fraud | flagged) — the exact inversion from 2.1, now with consequences. The diagnostic question: "the 92% was measured on known frauds. Do you know this transaction is a fraud?" No — that is what we are trying to find out. So the 92% is not our answer, it is one of our inputs.

2.3 The same model, moved to a riskier segment

A · M

Problem. The bank from 2.2 deploys the identical model — same 92% detection, same 4% false-positive rate — on a high-risk segment where 8% of transactions are fraudulent. Now what is the probability that a flagged transaction is fraudulent? Then state, in one sentence, what this proves about the model.

Solution. Same arithmetic, one number changed. Run it deliberately in parallel with 2.2 on the board.

Take 100,000 transactions again. 8% fraudulent is 8,000. Legitimate: 92,000.

True positives: 92% of 8,000 = 7,360. False positives: 4% of 92,000 — one percent is 920, so four percent is 3,680.

Total flagged: 7,360 + 3,680 = 11,040.

P(fraud | flagged) = 7,360 ÷ 11,040 = **0.667, about 67%.**

Now put the two answers side by side. 10.4% and 67%. Same model. Same detection rate, same false-positive rate, not one weight changed. The only difference is the population it was pointed at.

Trace the mechanism, because students should be able to explain it rather than just observe it. Two things moved together and both pushed the same way. The fraud group grew sixteen-fold, so true positives grew sixteen-fold — 460 to 7,360. And the legitimate group shrank slightly, so false positives fell a little — 3,980 to 3,680. The ratio between them swung enormously.

What this proves about the model: nothing. That is the sentence. The usefulness of a positive result is not a property of the model at all — it is a joint property of the model and the population. Move the same model to a different segment and the same alert carries completely different weight.

Which has a direct operational consequence worth stating to the class: this is the argument for segmenting before scoring. A single global fraud model serving both populations gives the analyst an alert whose meaning depends on which customer it came from — and the analyst has no way to know. Two models, or one model with segment-specific thresholds, gives alerts that mean the same thing every time.

Common mistake. Concluding "so the model is better on high-risk segments". It is not better; it is identical. What improved is the informativeness of its output. Students who blur those two will later try to fix a base-rate problem by retraining, which cannot work — you cannot train your way out of the arithmetic in 2.2.

2.4 Two words in a spam filter

M

Problem. 40% of incoming mail at a company is spam. The word "urgent" appears in 30% of spam and 5% of genuine mail. An email arrives containing "urgent". (a) What is the probability it is spam? (b) A second email contains both "urgent" and "wire", where "wire" appears in 20% of spam and 1% of genuine mail. Assuming the two words are independent given the class, what is the probability that email is spam?

Solution (a). Two routes to seeing "urgent", and we need both.

Spam *and* contains "urgent": 0.30 × 0.40 = 0.12. Note why we multiply — two conditions must both hold. The mail must be spam, and it must contain the word. Thirty percent of spam has it, but only forty percent of mail is spam, so we take thirty percent of that forty percent. In probability, "and" multiplies.

Genuine *and* contains "urgent": 0.05 × 0.60 = 0.03.

Total probability of seeing "urgent" at all: these are the only two ways it can happen, and they cannot both happen, so we add. 0.12 + 0.03 = 0.15. This is the denominator — the evidence term.

P(spam | "urgent") = 0.12 ÷ 0.15 = **0.80.**

Solution (b). Now both words. Independence given the class means: within spam, knowing "urgent" is present tells you nothing extra about "wire", so their probabilities multiply within each class.

Spam and both words: 0.30 × 0.20 × 0.40 = 0.024. Genuine and both words: 0.05 × 0.01 × 0.60 = 0.0003.

Total: 0.024 + 0.0003 = 0.0243.

P(spam | both) = 0.024 ÷ 0.0243 = **0.9877, about 98.8%.**

Look at what the second word did. From 80% to 98.8% — and notice it is the *genuine* branch that collapsed, from 0.03 to 0.0003, a factor of a hundred. Two words that are each individually weak evidence become overwhelming together, because a genuine email has to clear two unlikely hurdles at once.

This is exactly the mechanism of a naive Bayes classifier, and it is worth naming: "naive" refers precisely to the independence assumption we were handed. In real email "urgent" and "wire" almost certainly co-occur more than independence predicts, so the true posterior is somewhat lower than 98.8%. The assumption is wrong and the classifier works anyway — which is a genuinely useful thing for an ML engineer to have felt rather than been told.

Common mistake. Multiplying the two posteriors: 0.80 × something, or averaging them. Posteriors do not compose that way. Students must go back to the likelihoods, recombine, and renormalise. The fix is to insist on always rebuilding the two branches — spam-route and genuine-route — from scratch rather than reusing a previous answer.

2.5 A hallucination detector on a rare event

G

Problem. Your RAG assistant answers 20,000 questions a day. About 1.5% of its answers contain a fabricated claim. You deploy a detector that catches 85% of fabrications and falsely flags 6% of correct answers. Every flagged answer goes to a human reviewer, who takes 4 minutes per review. (a) How many reviews per day? (b) What fraction of a reviewer's time is spent on genuine problems? (c) How many reviewers do you need?

Solution (a). Same structure as 2.2, now with staffing attached.

Of 20,000 answers, 1.5% are fabrications: 0.015 × 20,000 = 300. Correct answers: 19,700.

True positives: 85% of 300 = 255. False positives: 6% of 19,700 — one percent is 197, so six percent is 1,182.

Total flagged: 255 + 1,182 = **1,437 reviews per day.**

Solution (b). Of those 1,437 reviews, 255 find a real fabrication. 255 ÷ 1,437 = 0.177.

About 18%. A reviewer spends roughly four minutes in five confirming that a perfectly good answer was perfectly good.

Solution (c). 1,437 reviews × 4 minutes = 5,748 minutes, which is 95.8 hours. At 6 productive review-hours per person per day — and do not use 8; nobody reviews for eight hours — that is 16 reviewers.

Sixteen full-time people, of whom the equivalent of thirteen are doing nothing useful. And note what we also learned in passing: 45 fabrications per day slip through undetected, because the detector catches 85% of 300.

Now the engineering consequence, which is why this is a GenAI problem rather than an arithmetic one. There are three levers and only one of them is the model.

Raise the detector's threshold: fewer false positives, fewer reviewers, more fabrications reaching users. Cheaper and riskier. Improve the detector's precision: expensive, slow, and the base rate still fights you. Or — the lever people forget — *narrow where the detector runs.* Apply it only to answers whose retrieval scores were weak, or that touch regulated topics. If that slice has a 10% fabrication rate rather than 1.5%, the same detector suddenly produces mostly-real alerts, exactly as in 2.3.

The third lever is almost always the right first move, and it is the only one that does not involve a trade-off you have to apologise for.

Common mistake. Computing 1,437 reviews and reporting the headcount without computing (b). The headcount alone reads as a capacity problem — "we need sixteen people" — and someone will hire them. The 18% figure reframes it as a targeting problem, which has much cheaper solutions. Always make students compute the precision of the alert queue before they staff it.

2.6 What you say in the credit committee

A · M · G

Problem. No arithmetic. A credit committee is shown a new default-prediction model. The slide says: "94% accurate. Flags 41% of applicants as elevated risk. We recommend auto-declining all flagged applications." The base rate of default in this portfolio is 5%. What do you say, and what do you propose instead?

Solution. Three things are wrong on that slide, and they should be raised in this order — weakest claim first, biggest consequence last.

First: the 94% is uninformative and should not be on the slide. With a 5% default rate, a model that approves everybody and flags nothing scores 95% accuracy. So this model, at 94%, is performing marginally worse than a rubber stamp on the metric being presented. That is not a subtle statistical point — it is a number anyone in the room can check in their head, and it means the headline claim carries no information about whether the model works.

Ask for precision and recall instead. Of the flagged applicants, what fraction actually defaulted? Of everyone who defaulted, what fraction did we flag?

Second: flagging 41% of applicants on a 5% base rate cannot be mostly right. You do not need the confusion matrix to know this — you need arithmetic. At most 5 percentage points of that 41 can be genuine defaulters, even if the model caught every single one. So the flagged pool is at best 5/41, about 12% real. Which means *at least* 88% of the auto-declines are creditworthy customers.

That bound is worth teaching as a reflex: **the flag rate cannot honestly exceed the base rate by much without most flags being false.** It lets you challenge a slide with no access to the underlying data.

Third, and this is the actual decision: auto-decline is the wrong action, and the model is not what makes it wrong. Even a well-calibrated posterior of, say, 38% default risk means the majority of those applicants would have repaid. Declining them turns a probability into a certainty of lost business.

What to propose. Route flagged applications to manual review rather than decline. Ask for the precision and recall figures before any threshold is fixed. And require the cost comparison the slide is missing: expected loss from a default versus expected lifetime margin from a good customer wrongly refused. Only after those two numbers exist can anyone choose a threshold rationally.

And the closing point for the class, which is the whole reason this problem has no arithmetic: the model was never the problem. The model produced a number. Somebody then chose an action, chose a metric to justify it, and chose not to mention the base rate. Every one of those was a human decision, and every one of them is where your judgement is required.

Common mistake. Attacking only the 94% figure and declaring victory. Correct but insufficient — a competent presenter will return next week with precision and recall and the same auto-decline recommendation. The base-rate critique wins the argument about the *metric*; the cost-asymmetry argument is what wins the argument about the *decision*, and the decision is what the committee is actually there to make.

TOPIC 03 · SIX PROBLEMS

Distributions & sampling

Choosing the wrong distribution is silent. The forecast simply never calibrates, and nobody can say why.

How many problems earn competence here: six. Four traps: choosing binomial when there is no n; forgetting the combination term in the binomial; treating a sample statistic as a fact; and believing that more data fixes bias. Two problems on distribution choice, two on the mechanics, two on sampling error — because the sampling half is the half that gets used in every meeting, and it is the half normally taught last and fastest.

3.1 Six questions, and which distribution each needs

A

Problem. For each, name the distribution and state what you would need to measure. (a) Of 12 kirana stores we onboarded this month, how many will still be active in a year? (b) How many UPI transactions will fail in the next hour? (c) How many of tomorrow's 500 support tickets will need escalation? (d) How many times will our API rate limit be breached this week? (e) What is the distribution of basket sizes at checkout? (f) Of 30 prompts in our regression suite, how many will pass after this model upgrade?

Solution. One question decides all six, and students should ask it out loud every time: *can I point at the trials and count them?*

(a) Binomial. Twelve stores. I could name them. Each either survives or does not — a yes-or-no outcome per trial, with n = 12. Need: n, and p, the historical one-year survival rate.

(b) Poisson. Now ask the question and watch it fail. How many trials were there? Was it one per second, so 3,600? One per millisecond? How many transactions *could* have failed? There is no answer — failures simply arrive from an unbounded pool. Need: λ only, the average failures per hour, measured from your own logs.

(c) Binomial. The 500 is given, so n exists. Need n and p, the historical escalation rate. Note the contrast with (b): both are about a fixed time window, but here somebody has told us the denominator.

(d) Poisson. Breaches arrive; nobody enumerated potential breaches. Need λ, average breaches per week.

(e) Neither — and this is the one worth spending time on. Basket size is not a count of successes among trials, nor of arrivals in a window. It is a quantity being measured per customer. That is a continuous-style distribution question, and in practice basket sizes are strongly right-skewed — most baskets small, a few enormous — so a normal distribution fits badly and the mean misleads. Need: the empirical distribution, and the median plus a high percentile rather than the mean.

(f) Binomial. Thirty prompts, each passes or fails. Need n = 30 and p. And flag the sequel now: with n = 30, the sampling error on the result will be enormous, which is problem 3.5.

Teach the pattern rather than the six answers. If the question names a denominator, it is binomial. If the question names a time window and no denominator, it is Poisson. If the question asks "how much" rather than "how many", neither applies.

Common mistake. Forcing (b) into a binomial by inventing an n — "let us say 10,000 possible transactions per hour". This looks harmless and quietly caps your forecast at 10,000, which reality never agreed to. It also makes p meaninglessly small, so the model becomes hypersensitive to a number you made up. If you cannot measure n, you do not have a binomial.

3.2 Exactly two churns out of six accounts

A · M

Problem. A B2B team manages 6 enterprise accounts up for renewal. Historically 25% churn at renewal. (a) What is the probability exactly 2 of the 6 churn? (b) What is the probability none churn? (c) What is the expected number of churns?

Solution (a). Binomial: n = 6, p = 0.25, k = 2. Build it in three pieces rather than substituting blindly, because the pieces are where the understanding lives.

Piece one — one specific pattern. Suppose accounts 1 and 2 churn and 3, 4, 5, 6 renew. Two churns at 0.25 each: $0.25^2 = 0.0625$. Four renewals at 0.75 each: 0.75^4. Compute that stepwise — $0.75^2 = 0.5625$, and $0.5625^2 = 0.3164$.

Why powers? Because each additional account is another independent condition that must hold, and independent conditions multiply. The exponent is literally a count of how many times you took a fraction of a fraction.

So one specific pattern: 0.0625 × 0.3164 = 0.01978.

Piece two — how many patterns satisfy us? We did not ask for accounts 1 and 2 specifically. Any two of the six churning counts. So: choose 2 positions from 6, order irrelevant. 6C2 = (6 × 5) ÷ 2 = 15.

And the reason we may simply multiply by 15 rather than adding fifteen separate cases: every one of those patterns has the identical probability, because each contains exactly two 0.25s and four 0.75s, and multiplication does not care about order.

Piece three. 15 × 0.01978 = **0.297, about 30%.**

Note the tension the class should feel: any one particular pattern was about a 2% event, yet the answer is 30%. Fifteen unlikely things, any of which satisfies you, sum to something quite likely.

Solution (b). None churn: k = 0. Only one pattern — everybody renews — so 6C0 = 1 and the combination term vanishes. $0.75^6 = 0.3164 \times 0.5625 =$ **0.178, about 18%.**

Solution (c). Expected number is n × p = 6 × 0.25 = **1.5 accounts.**

And make the class hold (b) and (c) together, because it corrects a common instinct. The expectation is 1.5, and the chance of the "everything is fine" outcome is only 18%. Losing zero accounts is the unusual case, not the normal one. A renewal plan built on "we'll probably keep them all" is planning for a one-in-six event.

Common mistake. Omitting the 6C2 and reporting 0.0198 — the probability of one named pair churning, not of any two. Diagnostic: ask the student to name which two accounts their answer refers to. If they cannot, the combination term is missing. Second frequent slip: computing 0.75^4 as 0.75 × 4.

3.3 Metro turnstile failures, and when to panic

A · M

Problem. A metro station averages 3 turnstile failures per day. (a) Probability of exactly 5 failures tomorrow? (b) Probability of zero failures? (c) Operations wants to alert whenever failures exceed the average. How often will that alert fire, and what should the threshold be instead?

Solution (a). Poisson — failures arrive, nobody counted potential failures. $\lambda = 3$, $k = 5$.

Three pieces. λ raised to k: $3^5 = 243$. Then k factorial: $5! = 120$. And e to the minus λ: $e^{-3} \approx 0.0498$.

Before multiplying, name what each piece does, because otherwise this is symbol-pushing. The 243 is roughly the weight of five events piling up. The 120 divides out the orderings of five interchangeable failures — you asked how many broke, not in what sequence, so the λ^5 term has over-counted by exactly 5!. Same instinct as dividing by r! in a combination. And e^{-3} is the normaliser that forces every possible k, from zero upward, to sum to one.

So: $0.0498 \times 243 \div 120$. Take it in order — $0.0498 \times 243 = 12.10$, and $12.10 \div 120 =$ **0.101, about 10%.**

Solution (b). $k = 0$. Anything to the power zero is 1, and $0! = 1$, so the whole expression collapses to $e^{-3} =$ **0.0498, about 5%.**

This is where $0! = 1$ earns its keep. Had it been zero, we would be dividing by zero and the formula would fail on the most ordinary question you could ask.

Solution (c). "Exceeds the average" means 4 or more. Let us compute how often that happens by finding the complement — the probability of 0, 1, 2 or 3.

$P(0) = 0.0498$. $P(1) = 0.0498 \times 3 = 0.1494$. $P(2) = 0.0498 \times 9 \div 2 = 0.2241$. $P(3) = 0.0498 \times 27 \div 6 = 0.2241$.

Sum: $0.0498 + 0.1494 + 0.2241 + 0.2241 = 0.6474$. So P(4 or more) = $1 - 0.6474 =$ **0.353.**

The alert fires on roughly 35% of days — about one day in three, forever. Within a fortnight nobody reads it, and the station has functionally lost its alerting.

What the threshold should be. Decide the tolerable false-alarm rate first, then find the count. For roughly one alert per month, we want a threshold whose exceedance probability is about 3%. Continuing the table: $P(4) = 0.1681$, $P(5) = 0.1008$, $P(6) = 0.0504$, $P(7) = 0.0216$, $P(8) = 0.0081$. Cumulative from 8 upward is about 1.1%; from 7 upward about 3.3%.

So alert at **7 or more failures** — roughly one alert a month, and each one genuinely unusual.

Also worth noting for a metro operator: 7 is more than double the average. People find that threshold counter-intuitively high, which is exactly why they set it at 4 and then drown.

Common mistake. Setting alert thresholds from the mean rather than from a tail probability. "Above average" is, by construction, roughly half of all days — it is not a definition of unusual, it is a definition of typical. Make students state the alert budget ("one per month") before they choose any number.

3.4 Staffing a support line at peak

A

Problem. A support line averages 8 calls per half-hour at peak. Each agent handles 2 calls per half-hour. (a) Staffing 4 agents covers the average — what fraction of half-hours will demand exceed capacity? (b) How many agents to cover 95% of half-hours? Use the Poisson tail values given: $P(X \geq 9) = 0.407$, $P(X \geq 11) = 0.184$, $P(X \geq 13) = 0.064$, $P(X \geq 14) = 0.034$, $P(X \geq 15) = 0.017$.

Solution (a). Four agents × 2 calls each = capacity of 8 calls. So demand exceeds capacity whenever 9 or more calls arrive. That is given: **0.407, about 41% of half-hours.**

This is the same lesson as 3.3(c) wearing a business suit. Staffing to the average means being overwhelmed roughly four half-hours in every ten — not occasionally, but as the standing condition. Anyone who has worked a support line will recognise the feeling and may never have known it was arithmetic.

Why does the average not cover you? Because arrival counts are variable, and for a Poisson process the variance equals the mean — so with $\lambda = 8$ the standard deviation is $\sqrt{8} \approx 2.8$. Fluctuations of ±3 calls are entirely ordinary, and each one is 1.5 agents' worth of work.

Solution (b). We want the probability of exceeding capacity to be at most 5%. Work backwards through the tail values.

Try 6 agents: capacity 12, so we fail when 13 or more arrive. $P(X \geq 13) = 0.064$ — that is 6.4%, still above our 5% budget. Close, but not there.

Try 7 agents: capacity 14, so we fail at 15 or more. $P(X \geq 15) = 0.017$, which is 1.7%. Comfortably inside budget.

Is 7 too generous? Check the intermediate: at capacity 13 we would fail at 14 or more, $P = 0.034 = 3.4\%$, which also meets the 5% target — but 13 is not achievable with whole agents at 2 calls each. So the answer is **7 agents.**

Now say the headline, because it is the point of the problem: covering the average takes 4 agents. Covering 95% of half-hours takes 7. **Nearly double the staff to handle variability, not volume.**

That ratio is why queueing is a discipline of its own, and why "we get 8 calls an hour on average" is not a staffing plan. It is also the honest argument for overflow routing and callbacks — buying the last few percent of coverage with headcount is extremely expensive, and there are cheaper instruments.

Common mistake. Dividing 8 by 2, answering "4 agents", and considering the problem solved. The arithmetic is right and the answer is wrong, because the question was about a distribution and the student answered about a mean. Watch for the same error in capacity planning, inventory and GPU provisioning — it is the identical mistake in three costumes.

3.5 Two models, thirty prompts, one promotion decision

M · G

Problem. Your regression suite has 30 prompts. Model A passes 21. Model B passes 24. A colleague wants to promote B. (a) What are the two pass rates? (b) What is the rough margin of error at n = 30? (c) Is the difference real? (d) How large would the suite need to be to resolve a 10-point difference?

Solution (a). 21 ÷ 30 = 70%. 24 ÷ 30 = 80%. A ten-point gap, which looks decisive on a slide.

Solution (b). Margin of error is roughly $1 \div \sqrt{n}$. $\sqrt{30} \approx 5.48$, so $1 \div 5.48 \approx 0.18$ — about **±18 percentage points.**

Which means Model A's true pass rate is somewhere around 52% to 88%, and Model B's is somewhere around 62% to 98%. Those two ranges overlap across most of their width.

Why the square root? Because errors are random and partly cancel. Adding thirty random errors does not give you thirty times one error — some are positive, some negative, and they largely destroy each other. What survives grows like $\sqrt{n}$ rather than n, so the error *per observation* shrinks like $1 \div \sqrt{n}$.

Solution (c). No. A ten-point gap measured with an eighteen-point instrument carries no information. The observed difference is entirely consistent with the two models being identical — and, uncomfortably, also consistent with A being better than B.

Make it concrete, because "not statistically significant" is a phrase people nod at without feeling. Three prompts separate these models. Three. Change three prompts in the suite — swap them for three others equally arbitrary — and the ranking could reverse. You are not measuring the models; you are measuring which thirty prompts somebody happened to write.

Solution (d). We want the margin of error down to about 5 points, so that a 10-point gap sits clear of the noise. $1 \div \sqrt{n} = 0.05$ gives $\sqrt{n} = 20$, so n = **400 prompts.**

Note the brutality of the square root: from 30 to 400 prompts, more than a thirteen-fold increase, and it only takes you from ±18 to ±5. To halve it again, to ±2.5, you would need 1,600 prompts.

Which yields the discipline worth teaching: decide the size of your evaluation set *before* you run the comparison. Ask "how small a difference must I be able to detect?", convert to n, and build that suite. Running first and arguing afterwards about whether the gap is real is how teams waste months and ship the worse model.

Common mistake. Treating "the difference is not significant" as "the models are the same". It is not a finding about the models — it is a finding about the measurement. You have learned nothing either way, which is a different and more honest statement than declaring equivalence.

3.6 Ten thousand evaluations that prove nothing

M · G

Problem. No arithmetic. A team evaluates their assistant on 10,000 real queries, drawn from the production logs of their three largest enterprise customers. Pass rate: 91%, with a margin of error under one point. They present this as evidence the assistant is ready for general release. What is wrong, and what would you ask for?

Solution. The statistics are impeccable. Ten thousand observations, margin of error about ±1 point — that is a genuinely precise measurement. And it may be almost worthless, for a reason no amount of additional data can repair.

The sample is biased, and bias does not shrink with n. This is the single most important idea in the sampling half of this topic, so it is worth stating carefully.

Random error and systematic error behave completely differently as you collect more data. Random error averages out — that is what the square-root law describes. Systematic error does not average out, because every additional observation carries the same distortion. Ten thousand queries from three large enterprise customers is a very precise measurement of what three large enterprise customers ask.

And "general release" means small customers, new customers, different industries, different vocabulary, users who phrase things badly, users in other languages, and — critically — the queries people *have not tried yet* because the product did not exist. None of those populations contributed a single observation.

Now the part that makes bias genuinely dangerous rather than merely unhelpful: the tight error bar makes the wrong answer look authoritative. At n = 30 nobody trusts the number, so nobody over-commits. At n = 10,000 with ±1 point, the figure acquires a credibility it has not earned, and it will be quoted in a board pack. **A confident wrong number is worse than an uncertain one.**

There is a second, subtler problem: production logs are a survivorship sample. They contain the queries people learned the product could handle. Every question a user tried once, got a poor answer to, and never asked again is absent from that log — and those are precisely the failures you needed to find.

What to ask for. Break the 91% out by customer — if it is 96%, 94% and 82%, the aggregate was hiding a segment that does not work. Add a deliberately adversarial set written by people who are not your users. Include queries the system is expected to *refuse*, since a log of successful use contains almost none. And ask what fraction of the 10,000 are near-duplicates: ten thousand queries from three customers may represent a few hundred distinct intents, which quietly reduces your effective sample size by an order of magnitude.

Closing line for the class: n tells you how precisely you measured. It says nothing at all about what you measured. Those are separate questions, and only one of them has a formula.

Common mistake. Proposing "collect more data" as the fix. More data from the same three customers gives a tighter estimate of the same biased quantity. The fix is a different sampling frame, not a bigger one — and students who miss this will spend real budget making a wrong number more precise.

TOPIC 04 · FIVE PROBLEMS

Vectors & embeddings

The arithmetic is school Pythagoras. The difficulty is entirely conceptual — what the numbers mean and what they cannot mean.

How many problems earn competence here: five. Only one mechanical skill — computing a distance — so one problem covers it and a second extends it to comparison. The remaining three exist because the traps are conceptual: believing a single embedding means something on its own, assuming dimensions are interpretable, and underestimating what dimensionality costs. Padding this to eight with more distance calculations would teach nothing; the arithmetic is learned after the second one.

4.1 Three customers, two features

A

Problem. Three customers are described by [monthly spend in thousands, visits per month]. P = [4, 3], Q = [8, 6], R = [5, 15]. (a) Which customer is closest to P? (b) What does that tell you, and what does it not?

Solution (a). Euclidean distance, one pair at a time.

P to Q: gaps are 8 – 4 = 4 and 6 – 3 = 3. Square them: 16 and 9. Sum: 25. Square root: **5.**

P to R: gaps are 5 – 4 = 1 and 15 – 3 = 12. Square: 1 and 144. Sum: 145. Square root: about **12.04.**

So Q is closer. And walk the class through why we square before adding, because it is not decoration. Two reasons. First, squaring removes sign — a gap of –4 is as much of a gap as +4, and without squaring, gaps in opposite directions would cancel and two genuinely different customers could score as identical. Second, Pythagoras is stated in squares: the theorem is $a^2 + b^2 = c^2$, not a + b = c. The square root at the end undoes the squaring and returns the answer to the original units, so "5" is a real straight-line distance rather than an abstract score.

Notice also which dimension decided the answer. P and R differ by only 1 on spend but by 12 on visits, and squaring amplified that: 144 against 1. The visits dimension made the entire decision. That is worth flagging — squaring means large gaps dominate, which is usually what you want and occasionally not.

Solution (b). What it tells you: among these three, Q is P's nearest neighbour, so if you were recommending an offer to P based on similar customers, you would look at Q.

What it does *not* tell you, and this is the important half. It does not tell you P and Q are similar in any absolute sense — 5 units apart is meaningless without a scale. If typical customers in this business are 40 units apart, then P and Q are near-twins. If typical customers are 2 apart, they are strangers. **A distance is only interpretable in comparison to other distances.**

It also does not tell you the two features deserve equal weight. Spend is in thousands of rupees and visits are a count — we implicitly declared that ₹1,000 of spend is worth exactly one visit by putting them on the same axes. Nobody decided that; it fell out of the units. Change spend to rupees rather than thousands and Q would be 4,000 units away on that axis, and the answer would flip. Raw-unit features almost always need scaling first.

Common mistake. Adding the gaps instead of the squared gaps — 4 + 3 = 7 for P to Q, and 1 + 12 = 13 for P to R. Here it happens to preserve the ranking, which is exactly why it is dangerous: the student gets the right answer by the wrong method and learns nothing. Ask them to draw the triangle; the hypotenuse is visibly not 7.

4.2 Deduplicating support tickets in three dimensions

M

Problem. Four support tickets have been embedded into three dimensions. T1 = [0.9, 0.2, 0.1], T2 = [0.8, 0.3, 0.1], T3 = [0.1, 0.9, 0.4], T4 = [0.2, 0.8, 0.3]. You want to cluster near-duplicates. (a) Compute all six pairwise distances. (b) Where would you set a duplicate threshold, and why there?

Solution (a). Six pairs. Work through them methodically — this is the problem where the mechanical skill becomes automatic.

T1–T2: gaps 0.1, −0.1, 0. Squares 0.01, 0.01, 0. Sum 0.02. $\sqrt{0.02} \approx$ **0.141.**

T3–T4: gaps −0.1, 0.1, −0.1. Squares 0.01, 0.01, 0.01. Sum 0.03. $\sqrt{0.03} \approx$ **0.173.**

T1–T3: gaps −0.8, 0.7, 0.3. Squares 0.64, 0.49, 0.09. Sum 1.22. $\sqrt{1.22} \approx$ **1.105.**

T1–T4: gaps −0.7, 0.6, 0.2. Squares 0.49, 0.36, 0.04. Sum 0.89. $\sqrt{0.89} \approx$ **0.943.**

T2–T3: gaps −0.7, 0.6, 0.3. Squares 0.49, 0.36, 0.09. Sum 0.94. $\sqrt{0.94} \approx$ **0.970.**

T2–T4: gaps −0.6, 0.5, 0.2. Squares 0.36, 0.25, 0.04. Sum 0.65. $\sqrt{0.65} \approx$ **0.806.**

Solution (b). Now sort the six numbers and look at the structure rather than the values: 0.141, 0.173, then a gap, then 0.806, 0.943, 0.970, 1.105.

That is the whole answer. There is a large empty region between 0.173 and 0.806 — nothing lives there. Two pairs are tightly close, four pairs are far apart, and no pair is ambiguous.

So set the threshold **in the gap** — anywhere around 0.4 or 0.5. T1 and T2 are duplicates of each other; T3 and T4 are duplicates of each other; the two clusters are unrelated.

And the reason to place it in the gap rather than at a value someone recommends: a threshold in an empty region is *robust*. Move it from 0.4 to 0.6 and not a single decision changes. A threshold at 0.15, by contrast, sits right between two real observations, and a tiny change in the embedding model would reclassify T3–T4.

Give the class the general method, since it is the transferable part: do not pick thresholds from intuition or from a blog post. Compute the distances you actually have, sort them, and look for the gap. If there is no gap, that is itself the finding — your data has no natural duplicate boundary, and any threshold you choose will be arbitrary and contested.

Common mistake. Setting the threshold just above the largest duplicate distance — here, 0.18 — on the reasoning that it "just captures" the known duplicates. This overfits to four data points. The next ticket pair that is genuinely duplicate at 0.25 will be missed. Aim for the middle of the gap, not the edge of the data.

4.3 Sizing a vector index before it is built

G

Problem. You are building retrieval over 2.4 million document chunks. You are choosing between a 384-dimension model and a 1,536-dimension model, both storing float32 (4 bytes per number). (a) Storage for each? (b) A brute-force search compares the query against every chunk — how many multiply-add operations per query, for each model? (c) The 1,536 model scores 3 points better on your retrieval evaluation of 200 queries. Do you take it?

Solution (a). Per vector: dimensions × 4 bytes. At 384 dims that is 1,536 bytes, call it 1.5 KB. At 1,536 dims it is 6,144 bytes, about 6 KB.

Across 2.4 million chunks: 2.4M × 1,536 bytes ≈ **3.7 GB** for the small model. And 2.4M × 6,144 ≈ **14.7 GB** for the large one.

Four times the dimensions, four times the storage — exactly linear, no surprises. But note where 14.7 GB sits: that no longer fits comfortably in memory on a modest server, which changes your infrastructure from "one box" to "a cluster or a disk-backed index". The cost is not the storage bill; it is the architecture.

Solution (b). Every comparison is a dot product over all dimensions — one multiply and one add per dimension.

At 384 dims: 2.4M × 384 ≈ **922 million operations per query.** At 1,536: 2.4M × 1,536 ≈ **3.7 billion per query.**

Again exactly four times. And this is why nobody runs brute-force search at this scale — you use an approximate index, which is a different subject. But the ratio survives approximation: whatever index you choose, the large model costs about four times as much per query.

Solution (c). No — or more precisely, you cannot tell yet, and 3 points on 200 queries is not evidence.

Apply topic 3. Margin of error at n = 200 is roughly $1 \div \sqrt{200} \approx 0.071$, so about ±7 percentage points. A 3-point difference measured with a 7-point instrument is noise. You would be paying four times the storage, four times the query compute, and a change of infrastructure, in exchange for a number that could easily reverse on a different 200 queries.

What to do instead: expand the evaluation set. To resolve 3 points you need the margin near 1.5 points, so $1 \div \sqrt{n} = 0.015$, giving $n \approx 4{,}400$ queries. That is a real amount of work — but it is far cheaper than a wrong permanent decision about your index.

And there is a third option students rarely propose: keep 384 dimensions and spend the saved compute on a re-ranking stage over the top 50 results. That usually buys more retrieval quality than a bigger embedding model, at lower cost. The dimensional upgrade is the obvious lever and frequently not the best one.

Common mistake. Computing the storage figures, seeing that 14.7 GB is "only a few hundred rupees a month", and concluding the upgrade is free. Storage is the cheapest of the three costs. Query latency and the infrastructure step-change are the expensive ones, and neither appears on a storage invoice.

4.4 What one dimension means

M · G

Problem. No arithmetic. A colleague inspects your embeddings and reports: "dimension 12 is high for all our complaint tickets and low for our praise tickets — dimension 12 is sentiment. Let us build a sentiment filter by thresholding it." Evaluate this proposal.

Solution. The observation may well be real. The conclusion does not follow, and the proposal will fail in a specific, predictable way.

First, why the observation is plausible. Embedding dimensions are not random noise — they encode learned structure. It is entirely possible that dimension 12 correlates strongly with sentiment in your corpus. Do not dismiss the colleague; they have noticed something.

Second, why "dimension 12 is sentiment" is the wrong description. Nobody assigned meanings to these axes. The model discovered a coordinate system that made its training objective work, and the axes it landed on are almost never aligned with human concepts. What typically happens is that sentiment is spread across many dimensions in combination, and dimension 12 happens to carry a chunk of it — along with several other things.

So the honest statement is: "dimension 12 correlates with sentiment on our current ticket set." Which is a much weaker claim, and correctly so.

Third, how it will fail. Three ways, and they are worth naming because each has been seen in production.

It will fail on new content. If dimension 12 also happens to encode, say, message length or formality — and complaints in your corpus are longer and more formal than praise — then the filter is a length detector wearing a sentiment badge. A short angry ticket sails through.

It will fail on a model upgrade. Retrain or switch the embedding model and the axes are re-derived from scratch. Dimension 12 becomes something else entirely — not shifted, unrelated. Your filter silently starts classifying on nothing, and there will be no error message. This is the failure that hurts most, because nothing breaks visibly.

And it will fail on subsets. The correlation was measured across your whole corpus; it may not hold within a particular customer's tickets or a particular language.

What to do instead: if you want sentiment, train a small classifier on the full embedding vector, using labelled examples. It will use dimension 12 if dimension 12 is informative, plus everything else that helps — and it will survive an embedding upgrade with a retrain rather than silently degrading. The general rule: **use the whole vector, let a model find the structure, and never build logic that depends on an individual dimension's identity.**

Common mistake. Rejecting the idea with "embeddings are uninterpretable" and moving on. That is too strong and it teaches the wrong lesson — the colleague's observation was probably genuine, and dismissing it discourages the kind of looking that finds real problems. The precise objection is about *depending* on an accidental axis alignment, not about whether structure exists.

4.5 Chunk size and the averaging problem

G

Problem. Your RAG system embeds whole documents as single vectors. An HR policy document covers leave, reimbursements, notice periods and dress code. A user asks "how many days notice do I need to give?" and retrieval fails to return this document, even though the answer is in it. Explain geometrically why, and give two fixes.

Solution. This is the most common structural failure in RAG systems, and it is entirely explainable with the geometry from this topic.

Why it fails. The document contains four unrelated topics. When you embed the whole thing as one vector, the model produces something like a blend — a position that represents "a document about several HR matters".

Now think about where that lands. If leave sits in one region of the space, reimbursements in another, notice periods in a third and dress code in a fourth, then the document's single vector sits somewhere near the middle of all four. And the middle of four regions is close to none of them.

The query "how many days notice" points firmly at the notice-period region. The document vector is not there — it is out in the averaged middle. So the distance is large, the similarity score is mediocre, and the document loses to some shorter, more focused chunk that is genuinely about only one thing.

Give the class the one-line version: **averaging several meanings produces a vector that means none of them.** A document about four things is not four times as findable; it is less findable than a document about one thing.

Fix one — chunk smaller. Split the document by section, so the notice-period paragraphs become their own vector sitting squarely in the notice-period region. The query now matches it directly. This is why chunking exists at all, and why "chunk on semantic boundaries" — headings, sections — beats "chunk every 500 characters", which cuts through the middle of ideas and produces exactly the blended vectors we are trying to avoid.

But be honest about the cost: chunk too small and each chunk loses its context. A paragraph reading "this must be submitted within 30 days" is a perfectly focused vector that is useless without knowing what "this" is. There is a genuine optimum, and it is what problem 1.4's chunk-size sweep was searching for.

Fix two — index at two levels. Keep small chunks for matching, but attach each chunk to its parent document, and return the surrounding context once a chunk is matched. You get the focused vector for retrieval and the full context for generation. This is usually the better answer in practice, because it stops treating "findable" and "useful to read" as the same requirement.

And a diagnostic worth teaching, since it detects this problem without any theory: if long documents in your corpus systematically retrieve worse than short ones, you have this failure. It is a two-minute query against your own index, and almost nobody runs it.

Common mistake. Diagnosing it as "the embedding model is bad at long documents" and switching models. The new model will do exactly the same thing, because averaging is not a defect of any particular model — it is what a single vector must do when asked to represent several unrelated ideas. The fix is in the chunking, not the model.

TOPIC 05 · FIVE PROBLEMS

Similarity & vector search

One formula, one idea — direction not length — and one number, the threshold, that quietly decides whether your system hallucinates.

How many problems earn competence here: five. One full cosine computation by hand — students must do it once, slowly, or the formula stays abstract. One on metric choice. One on reading scores correctly, which is where most misinterpretation lives. And two on the threshold, because the threshold is the highest-leverage number in a retrieval system and almost nobody is taught how to choose it.

5.1 Cosine similarity, every step, by hand

A · M

Problem. A query embeds to Q = [0.6, 0.8, 0.0]. Two documents: D1 = [0.3, 0.4, 0.0] and D2 = [0.0, 0.6, 0.8]. (a) Compute the cosine similarity of Q with each. (b) Compute the Euclidean distance of Q with each. (c) Which document should be returned, and what do the two metrics disagree about?

Solution (a). Take D1 first, and do every piece separately.

Dot product Q·D1: multiply matching components and add. (0.6 × 0.3) + (0.8 × 0.4) + (0.0 × 0.0) = 0.18 + 0.32 + 0 = 0.50.

Length of Q: $\sqrt{(0.36 + 0.64 + 0)} = \sqrt{1.00} = 1.0$. Convenient — Q is already a unit vector.

Length of D1: $\sqrt{(0.09 + 0.16 + 0)} = \sqrt{0.25} = 0.5$.

Cosine = 0.50 ÷ (1.0 × 0.5) = 0.50 ÷ 0.50 = **1.00.**

Perfect similarity. And students should see why before we move on: D1 is exactly Q multiplied by 0.5. Every component is half. Same direction, half the length. Cosine is deliberately blind to length, so it reports identity.

Now D2. Dot product Q·D2: (0.6 × 0.0) + (0.8 × 0.6) + (0.0 × 0.8) = 0 + 0.48 + 0 = 0.48.

Length of D2: $\sqrt{(0 + 0.36 + 0.64)} = \sqrt{1.00} = 1.0$.

Cosine = 0.48 ÷ (1.0 × 1.0) = **0.48.**

Look at where that 0.48 came from — entirely from the second dimension, the only one where both vectors are non-zero. On dimension one, Q has 0.6 and D2 has 0; product zero. On dimension three, Q has 0 and D2 has 0.8; product zero again. Two of the three dimensions contributed nothing, because agreement requires *both* vectors to be active. That is why we multiply rather than add — multiplication is a logical "and".

Solution (b). Euclidean distances.

Q to D1: gaps 0.3, 0.4, 0. Squares 0.09, 0.16, 0. Sum 0.25. $\sqrt{0.25}$ = **0.5.**

Q to D2: gaps 0.6, 0.2, −0.8. Squares 0.36, 0.04, 0.64. Sum 1.04. $\sqrt{1.04} \approx$ **1.02.**

Solution (c). Return D1 — both metrics agree on the ranking here, which is worth noticing, because they do not always.

What they disagree about is the *nature* of D1. Cosine says 1.00 — identical. Euclidean says 0.5 — not the same point. Both are correct, and they are answering different questions. Cosine asks "same topic?" and says yes, absolutely. Euclidean asks "same position?" and says no, D1 sits closer to the origin.

For text, cosine is the honest answer, because "closer to the origin" usually just means the document is shorter or the language is less emphatic — not that it is about something different. But the divergence is real information: a cosine of 1.00 with a distance of 0.5 is the signature of a short document on exactly your topic.

Common mistake. Forgetting to divide by the magnitudes and reporting the raw dot products — 0.50 and 0.48 — concluding the two documents are nearly equally relevant. Those raw numbers are close by coincidence: D1 has a small dot product because it is a short vector, not because it is a poor match. Dividing by length is exactly what corrects for that, and skipping it is the error cosine exists to prevent.

5.2 Four situations, two metrics

M · G

Problem. Cosine or Euclidean, and why? (a) Matching a two-line query against forty-page contracts. (b) Finding properties similar to a flat listed at ₹85 lakh, 1,200 sq ft, 4 km from the station. (c) Matching support tickets to knowledge-base articles, using an embedding model that returns unit-length vectors. (d) Clustering customers by [monthly spend, transaction count, account age in months].

Solution (a). Cosine. The length disparity is enormous — two lines against forty pages — and length is not what you are asking about. A contract is not less relevant to your query for being long. If you used Euclidean, every long document would sit far from every short query simply by virtue of having more of everything, and your two-line queries would preferentially retrieve two-line documents. This is the classic failure of early search engines.

Solution (b). Euclidean. And this is the case where cosine is actively wrong, so it deserves a full explanation.

Consider a flat at [85, 1200, 4] and another at [170, 2400, 8] — double on every axis. Cosine similarity: exactly 1.00, identical direction. So cosine declares a ₹1.7 crore, 2,400 sq ft property to be a perfect match for an ₹85 lakh flat.

It is not a match. It is twice the price. Here the magnitude *is* the signal — price and size are the substance of the comparison, not incidental scale. Cosine's blindness to magnitude, which was a feature in (a), is a catastrophe here.

One caveat to teach alongside: Euclidean on raw units means the ₹85 lakh figure dwarfs the 4 km figure, so price would dominate the distance entirely. Scale each feature first — divide by its standard deviation, or map to a 0–1 range — then use Euclidean.

Solution (c). Either — they give identical rankings. If every vector has length 1, then dividing by the magnitudes divides everything by 1, which changes nothing. More precisely, for unit vectors the two metrics are related by a fixed monotonic formula, so whichever ranks highest on one ranks highest on the other.

This is the practically valuable answer in this problem, because most embedding models return normalised vectors. Before your team spends an afternoon debating the metric, check whether the vectors are unit length. If they are, the debate is about nothing.

Solution (d). Euclidean, after scaling — same reasoning as (b). These are physical quantities in real units, and a customer spending ₹50,000 a month is genuinely different from one spending ₹5,000, not merely "the same shape, larger".

Give the class the decision rule: **ask whether doubling every number should mean "the same thing, bigger" or "a different thing".** Same thing bigger — a longer document on one topic — wants cosine. A different thing — a more expensive property, a bigger spender — wants Euclidean.

Common mistake. Answering "cosine" to all four, because cosine is the default in vector databases and therefore feels like the professional choice. The default exists because most vector-database traffic is text. The moment your features are physical measurements, the default is wrong — and students who have only ever done text retrieval will carry cosine into a pricing model and not notice.

5.3 Reading a retrieval score sheet

A · G

Problem. A query returns five chunks with cosine scores 0.91, 0.88, 0.42, 0.39, 0.37. A second query returns 0.61, 0.58, 0.55, 0.54, 0.52. For each query, describe what the score pattern tells you, and what the system should do.

Solution. Do not read the scores as percentages of relevance. Read the *shape* of the list. The gaps carry more information than the values.

Query one: 0.91, 0.88, then a cliff to 0.42.

Two documents are strongly on-topic and three are unrelated. The cliff between 0.88 and 0.42 is the useful signal — it says the corpus contains a clear answer and the retriever found it. Note that 0.42 is not "somewhat relevant": vectors at 0.42 are heading in substantially different directions, which means a different subject. A low cosine score means unrelated, not partially related.

What the system should do: pass the top two chunks to the model and discard the rest. Sending five chunks when three are noise wastes context and gives the model irrelevant material it may try to use. The cliff tells you exactly where to cut, per query, without a fixed k.

Query two: 0.61, 0.58, 0.55, 0.54, 0.52 — no cliff at all.

This pattern should worry you, and it is the more common of the two in real systems. Five documents all moderately similar, none clearly best. There are three plausible explanations and they need different responses.

First: the query is genuinely broad, so many documents are partially relevant. "What is our leave policy?" against a corpus with several overlapping HR documents would look like this.

Second: the corpus has no good answer, and these five are the least-bad among unrelated options. The flatness is the retriever having nothing to distinguish.

Third — and this is the one people miss — the chunks are too large, so every chunk is an average of several topics and they all land in the same mushy middle region. Exactly the averaging problem from 4.5. A flat score profile across many queries is the fingerprint of bad chunking.

What the system should do: without a cliff, there is no principled cut point. Take the top k and lower your confidence — or better, notice the flatness explicitly and either ask the user to narrow the question or state that the answer is assembled from several partial sources.

The transferable skill: log the top-score-minus-second-score gap, and the top-minus-fifth spread, for every query in production. Queries with a large gap are well-served. Queries with a flat profile are where your system is guessing — and that distribution, tracked over time, is a far better health metric than an average retrieval score.

Common mistake. Judging query two as healthier than query one because its worst score (0.52) is higher than query one's worst (0.37). The floor is irrelevant; the ceiling and the gap are what matter. Query one found a definite answer. Query two found five vague ones, which is a worse position to generate from.

5.4 Choosing the threshold from real data

G

Problem. You label 200 query-document pairs by hand as relevant or not, and record the cosine score of each. The counts are: at scores 0.8–1.0, 38 relevant and 2 irrelevant. At 0.6–0.8, 24 relevant and 11 irrelevant. At 0.4–0.6, 9 relevant and 47 irrelevant. At 0.2–0.4, 2 relevant and 67 irrelevant. (a) Compute the precision if you set the threshold at 0.4, at 0.6, and at 0.8. (b) Compute recall for each. (c) Which threshold, and on what grounds?

Solution. First, totals: relevant pairs are 38 + 24 + 9 + 2 = 73. Irrelevant are 2 + 11 + 47 + 67 = 127. Total 200. Keep 73 in mind — it is the denominator for every recall figure.

Threshold 0.8 — retrieve only the top band. Retrieved: 38 relevant + 2 irrelevant = 40.

Precision = 38 ÷ 40 = **0.95.** Recall = 38 ÷ 73 = **0.52.**

Almost everything you return is right, and you are missing nearly half of what exists.

Threshold 0.6 — top two bands. Retrieved: (38 + 24) relevant = 62, and (2 + 11) irrelevant = 13. Total 75.

Precision = 62 ÷ 75 = **0.83.** Recall = 62 ÷ 73 = **0.85.**

Threshold 0.4 — top three bands. Retrieved: 71 relevant, 60 irrelevant. Total 131.

Precision = 71 ÷ 131 = **0.54.** Recall = 71 ÷ 73 = **0.97.**

Solution (c). Look at the movement between thresholds rather than the absolute numbers, because the movement is where the decision lives.

Going from 0.8 down to 0.6: recall jumps from 0.52 to 0.85 — a gain of 33 points — while precision falls only from 0.95 to 0.83, a loss of 12. Excellent trade. You are buying a lot of coverage cheaply.

Going from 0.6 down to 0.4: recall gains 12 more points, from 0.85 to 0.97, while precision collapses from 0.83 to 0.54 — a loss of 29 points. Terrible trade. You are admitting 47 irrelevant documents to capture 9 relevant ones.

Choose 0.6. And notice this is visible in the raw data before any arithmetic: the 0.4–0.6 band is 9 relevant against 47 irrelevant, which is overwhelmingly junk. The band composition told you where the boundary was.

The general method, which is the point of the problem: label a couple of hundred pairs by hand — one afternoon of unglamorous work — bucket them by score, and read the precision-recall trade directly off the bands. This is how the single most important number in your retrieval system should be chosen. The alternative, which is what almost everyone does, is to guess 0.7 because it sounds reasonable.

Common mistake. Maximising F1 mechanically and stopping there. F1 at 0.6 is about 0.84 and at 0.4 about 0.69, so F1 agrees here — but F1 assumes precision and recall matter equally, which is a claim about your product, not a fact. For a legal-research assistant, missing a relevant precedent may be far worse than showing an extra irrelevant one, which argues for a lower threshold than F1 recommends.

5.5 The retriever that always returns three

G

Problem. No arithmetic. A user asks your internal assistant about a parental-leave policy that your company does not have and has never documented. Trace exactly what happens, step by step, and identify where the failure is introduced and where it should be fixed.

Solution. Walk it as a sequence, because the interesting part is that every component behaves correctly and the system still lies.

Step 1. The query is embedded. It becomes a perfectly good vector pointing at the parental-leave region of the space. Nothing wrong.

Step 2. The retriever searches the index and sorts by similarity. It is asked for the top three, so it returns three — perhaps a general leave policy at 0.44, a benefits summary at 0.41, an onboarding document at 0.38. All unrelated to parental leave.

And here is the crux: the retriever did not malfunction. It is a sorting machine. You asked for the best three and it gave you the best three. **Nothing in its design permits it to return an empty list.** It has no concept of "none of these are good enough" — that concept does not exist unless somebody adds it.

Step 3. The three chunks are assembled into a prompt, typically with an instruction like "answer the question using the context below".

Step 4. The model reads three documents about leave and benefits, and a question about parental leave, and does what it was instructed to do: it writes a fluent, confident, plausible answer about a policy that does not exist. It may well synthesise something reasonable-sounding from the general leave document.

Step 5. The user reads it and believes it, because it is well-written and cites internal documents.

Where the failure was introduced: step 2 — or more precisely, in the absence of a threshold check between steps 2 and 3. By step 4 the outcome was already determined. The model was handed irrelevant context and told to answer from it; producing something is the only behaviour consistent with its instructions.

Where it should be fixed: between steps 2 and 3. If the best score is below your threshold — 0.6, from problem 5.4 — do not call the model at all. Return "I do not have information on this." That is a few lines of code, it costs nothing, and it eliminates an entire class of hallucination.

Two things worth saying to the class about this. First, notice that no component was buggy. The embedder worked, the retriever worked, the model followed instructions. The failure lives in the *composition*, which is why it survives testing of individual parts. Second, this is why people who blame hallucination on the model are usually looking in the wrong place — a large fraction of production hallucinations are retrieval failures wearing a generation costume.

A useful secondary fix: instruct the model explicitly that it may answer "not found in the provided documents", and include examples of it doing so. That helps. But it is a second line of defence, not the first — you are asking the model to overcome your instruction to answer, which is a weaker guarantee than not asking it in the first place.

Common mistake. Locating the fix at step 4 — better prompting, a stricter system message, a bigger model. All of those are improvements and none is the fix, because the root cause is that irrelevant documents were admitted to the context in the first place. Prompt engineering to compensate for a missing threshold is treating a symptom at ten times the cost.

TOPIC 06 · THREE PROBLEMS

Graphs & relationships

A vocabulary topic with one genuine skill: recognising when a question is about relationships rather than resemblance.

How many problems earn competence here: three. The honest ceiling for this session. There is one trap — reaching for embeddings when the question is multi-hop — and one mechanical idea, the adjacency matrix. Three problems cover both with one to spare. Anything beyond that is teaching graph algorithms, which is a different course; padding this to six would be dishonest about how much of graph theory this audience actually needs.

6.1 Six questions: vector search or graph?

A · G

Problem. For each, say whether you would use vector similarity, a graph traversal, or both. (a) "Find me support tickets similar to this one." (b) "Which of our vendors are owned by the same parent company as this vendor?" (c) "Which employees would be affected if this manager left?" (d) "Find documents about the same subject as this one." (e) "Which accounts share a device with an account we just blocked, or share a device with an account that does?" (f) "What are the risks in this contract?"

Solution. One question decides it: *am I asking about resemblance, or about a chain of named relationships?*

(a) Vector. "Similar" is resemblance, which is exactly what a distance measures. No relationship is named.

(b) Graph. "Owned by the same parent" is a named relationship traversed twice — up from the vendor to its parent, then down to the parent's other subsidiaries. Embeddings cannot express this. Two vendors owned by the same parent might be in completely different industries with nothing similar about their descriptions, and vector search would never connect them. Conversely, two vendors with near-identical descriptions may be unrelated competitors — vector search would wrongly connect them.

(c) Graph. "Reports to" is a relationship, and the question is about following it — possibly several levels down, if you want the manager's whole subtree. Note that "affected" is doing quiet work here: it might mean direct reports only, or the entire reporting tree, and that ambiguity is a specification question, not a technical one.

(d) Vector. Same as (a). Resemblance.

(e) Graph, and this is the clearest case. Read the question again — "shares a device with an account that shares a device with a blocked account". That is explicitly two hops. There is no vector representation of "two hops away", because hop-count is a property of the connection structure, not of the accounts themselves. Two accounts can be one hop apart and be utterly dissimilar as objects.

(f) Both, and it is worth separating why. Vector search finds contract clauses resembling clauses known to be risky — that is resemblance. But a graph of clause dependencies tells you that clause 14 is only dangerous because clause 9 waived a protection, which is a relationship. Real contract analysis needs both, and this is a good illustration that "both" is a legitimate answer rather than a hedge.

The rule to carry: **if the question contains a named relationship — owns, reports to, shares, depends on, cites — and especially if it is applied more than once, it is a graph question.** If it contains "similar", "like this", or "about the same thing", it is a vector question.

Common mistake. Answering "vector, with a better embedding model" to (b) or (e). No embedding model, at any dimensionality, can encode "two hops along the shared-device edge", because that fact is not a property of either account — it is a property of the path between them. This is a limit of the representation, not of the model's quality, and students who do not see that will keep trying to solve graph problems by upgrading their embeddings.

6.2 An adjacency matrix, squared

M

Problem. Four accounts, W, X, Y, Z. W shares a device with X. X shares with Y. Y shares with Z. No other sharing. (a) Write the adjacency matrix. (b) Square it. (c) Interpret every entry of the result.

Solution (a). Rows and columns in order W, X, Y, Z. Put a 1 where an edge exists, 0 otherwise. Sharing is mutual, so the matrix is symmetric.

Row W: 0, 1, 0, 0 — connected to X only. Row X: 1, 0, 1, 0 — connected to W and Y. Row Y: 0, 1, 0, 1 — connected to X and Z. Row Z: 0, 0, 1, 0 — connected to Y only.

Note the diagonal is all zeros: no account shares a device with itself, which is a modelling choice worth making explicit rather than accidental.

Solution (b). Square it — multiply the matrix by itself. Each entry is a row dotted with a column.

Entry (W,W): row W [0,1,0,0] dotted with column W [0,1,0,0] = 0+1+0+0 = **1.**

Entry (W,X): row W dotted with column X [1,0,1,0] = 0+0+0+0 = **0.**

Entry (W,Y): row W dotted with column Y [0,1,0,1] = 0+1+0+0 = **1.**

Entry (W,Z): row W dotted with column Z [0,0,1,0] = **0.**

Continuing for the rest: row X gives 0, 2, 0, 1. Row Y gives 1, 0, 2, 0. Row Z gives 0, 1, 0, 1.

So A^2 is: [1,0,1,0], [0,2,0,1], [1,0,2,0], [0,1,0,1].

Solution (c). Every entry counts the *two-step* paths between that pair, and this is the payoff of the whole problem.

(W,Y) = 1: there is exactly one two-hop route from W to Y — via X. That is the fraud-ring question answered by arithmetic. W is not directly connected to Y, but they are two hops apart, and the matrix found it without anybody traversing anything.

(W,X) = 0: you cannot get from W to X in exactly two steps. One step yes, two steps no — W to X to W would return you to W, not X. Two-step paths of odd-distance pairs are zero, which is a nice structural fact.

(X,X) = 2: two two-step round trips from X — out to W and back, or out to Y and back. So the diagonal of A^2 counts each node's connections. X has degree 2, and there it is.

(W,Z) = 0: W and Z are three hops apart, so no two-step path exists. Cube the matrix and this entry becomes non-zero.

Say the general result plainly, because it is genuinely useful: **the entries of A^n count paths of length exactly n.** Which means multi-hop relationship questions are matrix multiplications — the same operation from Topic 7, applied to connections instead of features. This is precisely why graph neural networks are possible, and why graph algorithms and linear algebra keep turning out to be the same subject.

Common mistake. Reading A^2 entries as "is connected within two hops" rather than "number of paths of length exactly two". The distinction matters at (W,X) = 0: W and X *are* connected — directly — but not by a two-step path. For "within n hops" you need $A + A^2 + \ldots + A^n$, not A^n alone.

6.3 Why the fraud ring was invisible

A · M

Problem. No arithmetic. A bank runs a per-account fraud model on every account, scoring each on transaction patterns, age, KYC completeness and device history. It performs well on individual fraud. It completely misses a nine-account ring that operated for four months. Every account in the ring scored below the alert threshold. Explain why, and what would have caught it.

Solution. The model was asked the wrong question, and it answered that question correctly.

Why it missed. A per-account model consumes features of one account and outputs a score for that account. Every feature it looks at is a property of the account in isolation — its age, its transaction pattern, its documents.

Now consider what makes a ring a ring. Nine accounts, each individually unremarkable: plausible transaction volumes, complete KYC, ordinary ages. No single account does anything a legitimate account would not do. What is anomalous is the *pattern of connections between them* — money circulating in a closed loop, three shared devices, two shared addresses, accounts opened within days of each other.

None of that is a property of any account. It is a property of the structure. And a model whose input is one account's features literally cannot see it — the information is not in the input. This is not a threshold problem, a training-data problem or a model-capacity problem. The signal was never presented.

That is the essential point for the class: **you cannot detect a relational pattern from non-relational features, at any model quality.** Individually innocent, collectively obvious.

What would have caught it. Build the graph. Accounts as nodes; edges wherever two accounts share a device, an address, a phone number, or transact with each other. Then look for structure rather than for suspicious individuals.

Specifically, three signals. Unusual density — nine accounts with a dozen shared attributes between them is far more interconnected than nine random accounts, which would typically share nothing. Closed loops — money returning to its origin through a cycle, which almost never happens in legitimate commerce and is easy to detect on a directed graph. And temporal clustering on edges — accounts that appeared within days of each other and immediately connected.

Then feed those graph features *back into the per-account model* as additional inputs: "number of accounts sharing a device with this one", "size of the connected component this account sits in", "is this account part of a cycle". Now the relational information is in the input, and the existing model can use it.

That is usually the practical answer, and it is worth stating because students tend to imagine an either/or choice: you rarely replace the per-account model with a graph model. You compute graph features and give them to the model you already have.

Common mistake. Proposing "lower the alert threshold" so the nine accounts get flagged. Their scores were low because their individual behaviour was genuinely unremarkable — lowering the threshold to catch them would flood the queue with thousands of equally unremarkable legitimate accounts, at exactly the precision cost computed in problems 2.2 and 2.5. Missing information cannot be recovered by moving a threshold.

TOPIC 07 · SIX PROBLEMS

Matrices & tensors

The most reusable topic in the session, and the one where students are most likely to be able to compute without understanding.

How many problems earn competence here: six. Unusually, the mechanics genuinely need repetition — students can follow a worked 2×2 multiplication and still be unable to start one, so two problems go on multiplication alone. Then shapes, which is where every real error lives; the transpose, because attention depends on it; and a memory-sizing problem, because tensor shapes are how you predict cost. Six is the floor here rather than the ceiling — this is the one topic where eight would be defensible if time allowed.

7.1 Scoring four applicants against one rule

A · M

Problem. Four loan applicants, three features each — [income score, credit score, repayment score]. A = [80, 70, 90], B = [60, 85, 75], C = [95, 50, 60], D = [70, 75, 80]. The weight vector is [0.5, 0.3, 0.2]. (a) Compute all four scores. (b) State the shapes involved. (c) Which applicant benefits most from these particular weights, and what would change if the weights were [0.2, 0.3, 0.5]?

Solution (a). Each score is one row dotted with the weight vector: multiply each feature by its weight, add the results.

A: (0.5 × 80) + (0.3 × 70) + (0.2 × 90) = 40 + 21 + 18 = **79.**

B: (0.5 × 60) + (0.3 × 85) + (0.2 × 75) = 30 + 25.5 + 15 = **70.5.**

C: (0.5 × 95) + (0.3 × 50) + (0.2 × 60) = 47.5 + 15 + 12 = **74.5.**

D: (0.5 × 70) + (0.3 × 75) + (0.2 × 80) = 35 + 22.5 + 16 = **73.5.**

Ranking: A, C, D, B.

Note why we multiply feature by weight — a weight is a claim about importance, and importance scales a contribution. And why we add rather than multiply the results: addition lets strengths compensate for weaknesses. Under multiplication, a single zero would annihilate the whole score, meaning any one weak feature disqualifies the applicant outright. Occasionally you want that; usually you do not.

Solution (b). The applicant matrix is (4 × 3) — four rows, three features. The weight vector is (3 × 1). So (4 × 3) × (3 × 1) → (4 × 1). The inner 3s match and vanish; the outer 4 and 1 survive. Four applicants in, one column of four scores out.

And the vanished 3 is worth naming: it was the length of each dot product — the work done — and it leaves no trace in the shape of the answer. That is where the computation went.

Solution (c). C benefits most from these weights. Look at C's profile: 95 on income, but only 50 and 60 elsewhere. C is a one-strength applicant, and income happens to carry the largest weight. Under a fairer reading of the numbers C is the weakest applicant here — lowest total raw score at 205, against A's 240 — yet C ranks second.

Now flip the weights to [0.2, 0.3, 0.5], emphasising repayment history. A: 16 + 21 + 45 = 82. B: 12 + 25.5 + 37.5 = 75. C: 19 + 15 + 30 = 64. D: 14 + 22.5 + 40 = 76.5.

New ranking: A, D, B, C. C has fallen from second to last, and B and D have both overtaken.

Same applicants, same data, same arithmetic — a completely different lending decision, because someone chose different weights. Worth saying explicitly: when a rejected applicant asks "why was I declined?", the honest answer is as much about the weights as about them. That is not a mathematical fact; it is a governance one.

Common mistake. Averaging the three features instead of applying the weights — giving A 80, B 73.3, C 68.3, D 75. That is a valid calculation of a different quantity, and it silently asserts that all three features matter equally. Students should notice that an unweighted average *is* a weighted average with weights [⅓, ⅓, ⅓]; there is no such thing as "no weighting".

7.2 Two matrices, both orders

M

Problem. Let P = [[2, 1], [0, 3]] and Q = [[4, 5], [6, 7]]. (a) Compute P × Q. (b) Compute Q × P. (c) Explain, in terms of what the matrices represent, why the two answers differ.

Solution (a). Four entries, each a row of P dotted with a column of Q. Walk across the row, down the column.

Row 1 of P is [2, 1]. Column 1 of Q is [4, 6] — read *down*, not across; this is where most errors start.
(2 × 4) + (1 × 6) = 8 + 6 = **14.**

Row 1 × Column 2, which is [5, 7]: (2 × 5) + (1 × 7) = 10 + 7 = **17.**

Row 2 of P is [0, 3]. × Column 1: (0 × 4) + (3 × 6) = 0 + 18 = **18.**

Row 2 × Column 2: (0 × 5) + (3 × 7) = 0 + 21 = **21.**

P × Q = **[[14, 17], [18, 21]].**

Solution (b). Now Q × P. Rows come from Q, columns from P.

Row 1 of Q is [4, 5]. Column 1 of P is [2, 0]: (4 × 2) + (5 × 0) = 8 + 0 = **8.**
Column 2 of P is [1, 3]: (4 × 1) + (5 × 3) = 4 + 15 = **19.**

Row 2 of Q is [6, 7]. × Column 1: (6 × 2) + (7 × 0) = **12.**
× Column 2: (6 × 1) + (7 × 3) = 6 + 21 = **27.**

Q × P = **[[8, 19], [12, 27]].**

Not one entry matches. This is not a small perturbation — it is a different answer.

Solution (c). The roles are asymmetric, and naming them explains everything.

In P × Q, the rows of P are the things being scored and the columns of Q are the scoring rules. In Q × P, the rows of Q are being scored using columns of P as rules. The objects and the operators have swapped jobs. Of course the answer differs — it is a different question.

Or, in transformation language: matrix multiplication composes transformations, and composition has an order. Rotating then stretching is not the same as stretching then rotating — try it with your hands.

Which is exactly why the order of layers in a network matters and why you cannot rearrange them for convenience. Passing data through layer one and then layer two is a genuinely different computation from the reverse — like washing then drying your clothes versus drying then washing them.

Common mistake. Reading across the second matrix instead of down its columns — pairing row [2,1] with row [4,5] to get 8 + 5 = 13. This is by far the most frequent mechanical error, and it is why students should physically trace the row with one finger and the column with another for the first several problems. The muscle memory matters more than the explanation.

7.3 Six shape questions, and one error message

M · G

Problem. (a) (16 × 32) × (32 × 8) → ? (b) (100 × 5) × (5 × 5) → ? (c) (7 × 3) × (7 × 3) → ? (d) A batch of 128 embeddings of 1,024 dims, times a weight matrix (1024 × 4096) → ? (e) Given (64 × 512) and (64 × 512), produce a (64 × 64) result — how? (f) You see: "mat1 and mat2 shapes cannot be multiplied (256×768 and 1024×768)". What is the fix?

Solution. One rule throughout: **(m × n) × (n × p) → (m × p).** Inner numbers must match and disappear; outer numbers survive.

(a) Inner: 32 and 32. Match. Result **(16 × 8).**

(b) Inner: 5 and 5. Result **(100 × 5).** Note the shape is unchanged — a square matrix on the right transforms each row in place without changing how many numbers describe it. This is what most layers inside a transformer block look like.

(c) Illegal. Inner numbers are 3 and 7. No pairing exists — you would walk along a row of 3 and down a column of 7, and after three multiplications you would be holding four numbers with nothing to multiply them by. The operation is not wrong, it is undefined, which is why frameworks refuse rather than approximate.

(d) (128 × 1024) × (1024 × 4096) → **(128 × 4096).** A hundred and twenty-eight items came in described by 1,024 numbers and left described by 4,096. The layer expanded the representation — this is the "up-projection" in a feed-forward block, and it is where a large share of a model's parameters live.

(e) Transpose the second one. (64 × 512) × (512 × 64) → (64 × 64). And notice what that result means: 64 rows compared against 64 rows, one score per pair. That is the attention pattern — this is precisely $Q \times K^T$, and the transpose is what makes "every item against every item" expressible at all.

(f) Read the inner numbers: 768 and 1024. Mismatch. But look closer — 768 appears in *both* matrices, just in the wrong position in the second. That tells you nothing is architecturally wrong; the grid is merely oriented the wrong way.

Transpose the second matrix: (256 × 768) × (768 × 1024) → **(256 × 1024).** Legal.

Teach the diagnostic as a two-step reflex, because students will use it within a week of touching real code. First, extract the two inner numbers. Second, ask whether the shared dimension appears in both shapes. If it does, you need a transpose. If it does not, you have a genuine design error — the two things you are multiplying were never compatible, and no rearrangement will save it.

Common mistake. Reflexively transposing whichever matrix makes the error go away, without checking that the shared dimension genuinely exists in both. If the shapes are (256 × 768) and (1024 × 512), transposing produces a legal multiplication of two things that have no business being multiplied — the code runs and the numbers are meaningless. A silent wrong answer is worse than a crash.

7.4 Attention scores for three tokens

M · G

Problem. Three tokens with two dimensions each. Q = K = [[2, 0], [0, 2], [1, 1]] — call the rows "the", "cat", "sat". (a) Write K^T and confirm the shapes work. (b) Compute Q × K^T in full. (c) Interpret the matrix. (d) Why is the result divided by $\sqrt{d}$ before softmax?

Solution (a). Q is (3 × 2) — three tokens, two dimensions. K^T flips it to (2 × 3): columns become [2, 0, 1] on the first row and [0, 2, 1] on the second. So K^T = [[2, 0, 1], [0, 2, 1]].

Shapes: (3 × 2) × (2 × 3) → (3 × 3). Nine scores for three tokens — one per ordered pair. Without the transpose we would have (3 × 2) × (3 × 2), inner numbers 2 and 3, illegal.

Solution (b). Nine entries. Row of Q dotted with column of K^T.

Row "the" = [2, 0]. Against column 1 [2, 0]: 4 + 0 = **4.** Against column 2 [0, 2]: 0 + 0 = **0.** Against column 3 [1, 1]: 2 + 0 = **2.**

Row "cat" = [0, 2]. Column 1: 0 + 0 = **0.** Column 2: 0 + 4 = **4.** Column 3: 0 + 2 = **2.**

Row "sat" = [1, 1]. Column 1: 2 + 0 = **2.** Column 2: 0 + 2 = **2.** Column 3: 1 + 1 = **2.**

Result: **[[4, 0, 2], [0, 4, 2], [2, 2, 2]].**

Solution (c). Read it row by row — each row is one token's attention profile.

"The" scores 4 with itself, 0 with "cat", 2 with "sat". It is strongly self-focused and has literally nothing in common with "cat" — they are active on different dimensions, so every product is zero. Same story for "cat", mirrored.

"Sat" is the interesting row: 2, 2, 2. Perfectly flat. Because it is active on both dimensions at half strength, it overlaps equally with everything — including itself. It attends broadly and to nothing in particular.

Notice the diagonal is largest for "the" and "cat" but not for "sat". A token whose vector is spread across dimensions does not stand out even against itself. And notice the matrix is symmetric here only because Q = K; in a real transformer Q and K come from different learned projections, so the matrix is asymmetric — "cat" may attend strongly to "the" without the reverse being true.

Solution (d). Dividing by $\sqrt{d}$ — where d is the dimension count — keeps the scores in a range where softmax behaves usefully.

Here is the mechanism. A dot product sums d products, so its typical magnitude grows with d. At d = 2 our scores are 0 to 4. At d = 768, comparable vectors produce dot products in the hundreds.

Feed scores of, say, 300 and 280 into softmax and exponentiate: e^{300} against e^{280} is a ratio of e^{20}, about 500 million to one. The distribution becomes a hard maximum — one token gets essentially all the attention, everything else gets zero. That is bad in two ways: the model loses the ability to blend information, and the gradients through those near-zero terms vanish, so it cannot learn.

Dividing by $\sqrt{d}$ cancels exactly the growth that d introduced, so scores stay in a range where the gaps are meaningful and the softmax stays soft. It is scalar multiplication — the simplest operation in the topic — doing something essential.

Common mistake. Believing the $\sqrt{d}$ division exists to prevent numerical overflow. It does not — the max-subtraction trick from Topic 12 handles overflow. This is about keeping the distribution soft enough that attention blends and gradients flow. Two different problems with two different fixes, and students conflate them constantly.

7.5 Sizing a batch before it runs out of memory

M · G

Problem. You are serving a model with 512-token context and 1,024 hidden dimensions, in bfloat16 (2 bytes per number). (a) How many numbers in one batch of 16 sequences of activations, and how much memory? (b) Attention builds a score matrix per sequence — how many numbers is that for the batch, and how does it scale with context length? (c) You double the batch to 32 and the context to 1,024. What happens to each of the two memory figures?

Solution (a). The activation tensor has shape (batch × tokens × dimensions) = (16 × 512 × 1024).

Multiply: 16 × 512 = 8,192. Then 8,192 × 1,024 = **8,388,608 numbers** — about 8.4 million.

At 2 bytes each: 16.8 million bytes, or roughly **16 MB.**

Modest. And that is for one layer's activations — a 32-layer model holding activations for a backward pass would need roughly 32 times that, about 512 MB, which is where the number starts to matter.

Solution (b). The attention score matrix compares every token with every other token, so per sequence it is (tokens × tokens) = 512 × 512 = 262,144 numbers.

Across the batch of 16: 16 × 262,144 = **4,194,304 numbers**, about 4.2 million, or **8 MB** at 2 bytes.

Now the scaling, which is the point of this part. The activation tensor grows *linearly* with context length — double the tokens, double the numbers. The score matrix grows with tokens *squared*, because every token must be compared with every token. Double the tokens and it quadruples.

That quadratic term is the single most important cost fact about transformers, and it is visible directly in the shape.

Solution (c). Batch 16 → 32, context 512 → 1,024.

Activations: (32 × 1024 × 1024). Batch doubled (× 2) and tokens doubled (× 2), so 4 times the original. 16 MB → **64 MB.**

Score matrices: per sequence now 1,024 × 1,024 = 1,048,576 — four times, from the squaring. And 32 sequences instead of 16 — another factor of 2. Total factor of 8. 8 MB → **64 MB.**

Both land at 64 MB here, which is a nice coincidence to point at: the attention term started at half the activation cost and has caught up. Extend the context again to 4,096 and attention would be sixteen times the activation cost. That crossover is why long-context models required architectural work — flash attention, sliding windows, and so on — rather than just more memory.

The transferable habit: before running anything, multiply the shape out. Three numbers and one multiplication tell you whether your configuration fits, and which term will dominate as you scale. Nobody should discover a memory limit by hitting it.

Common mistake. Assuming that doubling the context doubles the memory. It is true for the activations and false for attention, and the attention term is the one that eventually dominates. Students who carry the linear intuition will size a long-context deployment by a factor of two and be wrong by a factor of eight.

7.6 Reading a model card's parameter count

M · G

Problem. A model has 32 layers. Each layer contains four attention projection matrices of shape (4096 × 4096), and two feed-forward matrices of shapes (4096 × 11008) and (11008 × 4096). (a) Parameters per layer? (b) Total across all layers? (c) Memory in fp16 and in int4. (d) The vocabulary embedding is (32000 × 4096) — how much does that add, and what does its size tell you?

Solution (a). A matrix's parameter count is simply rows × columns — every entry is one learned number.

Attention: four matrices at 4,096 × 4,096 = 16,777,216 each. Times four: 67,108,864, about **67 million.**

Feed-forward: 4,096 × 11,008 = 45,088,768. The second matrix has the same product reversed, so another 45,088,768. Together about **90 million.**

Per layer: 67 + 90 = **about 157 million parameters.**

Note the split — the feed-forward block holds more parameters than attention, roughly 57% against 43%. Attention gets all the attention, and the majority of the weights are in the plainest part of the block.

Solution (b). 157 million × 32 layers = **about 5.0 billion parameters.**

Solution (c). fp16 is 2 bytes per parameter: 5.0 billion × 2 = **10 GB.** int4 is half a byte: 5.0 billion × 0.5 = **2.5 GB.**

Ten gigabytes needs a serious GPU. Two and a half fits on a laptop. Same model, same learned behaviour, one decision about number format — and that is the entire business case for quantisation, computed from a model card in four multiplications.

Solution (d). The embedding matrix is 32,000 × 4,096 = 131,072,000, about **131 million parameters** — 262 MB in fp16.

What its size tells you: it is roughly 2.6% of the model, and it is about 83% of a single layer. So the entire vocabulary — every token the model knows, in every language — costs less than one transformer layer. Knowledge is not stored in the vocabulary; it is stored in the layers.

And a practical corollary worth mentioning: expanding the vocabulary is cheap in parameters, which is why adding tokens for a new language or domain is a real option. Adding layers is expensive. The parameter arithmetic tells you which levers are affordable before anyone opens a training script.

Common mistake. Confusing parameter memory with total memory needed to run the model. Weights are only one term — activations (problem 7.5), the KV cache, optimizer state during training, and gradients all add to it. A 10 GB model does not fit in 10 GB of VRAM. For inference, budget roughly 1.2 to 1.5 times the weights; for training, several times more.

TOPIC 08 · FOUR PROBLEMS

Logs & exponentials

A supporting topic. Nobody's job is logarithms — but four other topics break without them.

How many problems earn competence here: four. This topic exists to serve softmax, cross-entropy, log-likelihood and scaling charts. So the problems are chosen to be exactly the four uses: turning a product into a sum, reading a negative log-probability without alarm, understanding why e^x is always positive, and reading a log-scale chart. There is no independent skill to drill — a fifth problem would be arithmetic practice with no destination.

8.1 A sentence probability that underflows

M

Problem. A model assigns these probabilities to four consecutive tokens: 0.5, 0.1, 0.01, 0.2. (a) What is the probability of the whole sequence? (b) Express it as a sum of logs, base 10. (c) If a 600-token sentence averaged 0.1 per token, what is the sequence probability, and what happens in a computer? (d) What is the log-probability instead?

Solution (a). The tokens are consecutive, and each must follow the last, so all four conditions must hold. "And" multiplies.

0.5 × 0.1 = 0.05. × 0.01 = 0.0005. × 0.2 = **0.0001**, or 10^{-4}.

Four tokens and we are already at one ten-thousandth. Every factor is below 1, so the product only ever shrinks.

Solution (b). Take logs base 10 of each: log(0.5) ≈ −0.301, log(0.1) = −1, log(0.01) = −2, log(0.2) ≈ −0.699.

Sum: −0.301 − 1 − 2 − 0.699 = **−4.0.**

And check it: 10^{-4} is 0.0001, exactly our answer from (a). The log of the product equals the sum of the logs, and both routes agree.

Why does that identity hold? Count multiplications. A logarithm answers "how many times do I multiply the base to get here". If one number needs two tens and another needs three, then their product needs five. You are adding exponents, which is the oldest rule in the book.

Solution (c). 0.1 to the power 600 is 10^{-600}.

In a computer, this is **zero.** A 32-bit float bottoms out around 10^{-38}, and even a 64-bit double around 10^{-308}. We are hundreds of orders of magnitude past both.

And once it is zero, the damage is total rather than partial. Every long sentence scores zero — a beautiful one and a nonsensical one, identically. The model has not become less accurate; it has become completely blind, because it can no longer distinguish between any two long sequences. You cannot rank by a quantity that is always zero.

Solution (d). In logs: 600 × log(0.1) = 600 × (−1) = **−600.**

Minus six hundred. A number a computer holds with total comfort and complete precision. Nothing was lost, every comparison still works, and a sequence scoring −590 is correctly identified as better than one scoring −610. Same information, no collapse — and that is the entire reason logs appear throughout machine learning.

Common mistake. Treating underflow as a rounding inconvenience — "we lose a little precision". It is not a precision loss, it is a total loss of ordering. Have students state what happens when two different sentences both score zero: the model cannot prefer either, so generation, ranking and beam search all fail simultaneously.

8.2 Reading logprobs from an API

G

Problem. An API returns these natural-log probabilities for four candidate tokens: −0.11, −2.30, −4.61, −6.91. (a) Convert each to a probability. (b) Do they sum to 1, and what does that tell you? (c) A colleague says "the −6.91 means the model is 6.91 units wrong". Correct them. (d) Which is more confident: a top logprob of −0.05 or −0.69?

Solution (a). Exponentiate: probability = e raised to the logprob. Use the landmarks — $e^{-2.30} \approx 0.10$ and $e^{-4.61} \approx 0.01$, because ln(10) ≈ 2.303, so every −2.303 is another factor of ten.

$e^{-0.11} \approx$ **0.90.** $e^{-2.30} \approx$ **0.10.** $e^{-4.61} \approx$ **0.010.** $e^{-6.91} \approx$ **0.001.**

Teach the landmark rather than the arithmetic: **each −2.3 in a natural logprob is a factor of ten.** With that one fact you can read logprobs at a glance for the rest of your career.

Solution (b). 0.90 + 0.10 + 0.010 + 0.001 = 1.011.

Slightly over 1, which is rounding — but the important point is that they sum to *approximately* 1, and that tells you something. These four are essentially the whole distribution. The remaining 99,996 tokens in the vocabulary share almost nothing between them.

If instead they had summed to 0.4, you would know the model was genuinely uncertain, with 60% of its probability spread across tokens the API did not show you. Always check what the top-k logprobs sum to — it is a free measure of how much you are not seeing.

Solution (c). Two things to correct.

First, a logprob is not an error measurement. It is a probability, expressed in logs. −6.91 means "this token has probability 0.001" — the model is not wrong about anything; it is reporting a small likelihood.

Second, the negative sign is not a signal of badness. *All* logprobs are negative, necessarily. Probabilities are below 1, and the log of any number below 1 is negative. A logprob of 0 would mean probability exactly 1, which never happens in a softmax. So seeing negative numbers is not information — seeing *how* negative is.

Solution (d). −0.05 exponentiates to about 0.95. −0.69 exponentiates to about 0.50 — worth memorising, since ln(0.5) ≈ −0.693.

So −0.05 is far more confident: 95% against 50%. And note how small the difference looks on the log scale — 0.64 apart — while the probabilities differ by 45 points. Log scales compress, which is their purpose, and it means small-looking logprob differences near zero are large probability differences. Do not eyeball logprobs; convert them.

Common mistake. Averaging logprobs across a sequence and reporting the result as "the model's confidence in the answer". Averaging in log space is a geometric mean of probabilities, not an arithmetic one, and it systematically punishes long outputs — one unlikely token drags the whole average. If you want a length-fair confidence measure, be explicit about which one you are computing and why.

8.3 Why e^x and not something simpler

M

Problem. A model produces four raw scores: 3.0, 1.0, 0.0, −2.0. You need to convert them into probabilities. (a) Why can you not simply divide each by the total? (b) Compute e^x for each and confirm all are positive. (c) Convert to probabilities. (d) Compare against a naive alternative: shift everything up by 2 to remove negatives, then divide by the total. What differs, and which behaviour do you want?

Solution (a). Try it and watch it fail. The total is 3.0 + 1.0 + 0.0 − 2.0 = 2.0.

Dividing: 3.0 ÷ 2.0 = 1.5. Already broken — a probability of 1.5 does not exist. And the last one gives −2.0 ÷ 2.0 = −1.0, a negative probability. Two of the four results are outside [0, 1].

The problem is that raw scores can be negative, and dividing does nothing about signs. Worse, if the scores happened to sum to zero you would be dividing by zero. Division alone cannot produce a valid distribution from arbitrary numbers.

Solution (b). $e^3 \approx 20.09$. $e^1 \approx 2.718$. $e^0 = 1.000$. $e^{-2} \approx 0.135$.

All positive — including the one that came from a negative input. And that is guaranteed, not lucky: e^x is repeated multiplication of a positive number, and no amount of multiplying or reciprocating a positive number reaches zero, let alone crosses it. e^{-1000} is unimaginably small and still positive.

Note also that the ordering survived: 3.0 > 1.0 > 0.0 > −2.0 became 20.09 > 2.718 > 1.000 > 0.135. Exponentiating never reshuffles the ranking, which matters — you want to fix the range without changing which token wins.

Solution (c). Sum: 20.09 + 2.718 + 1.000 + 0.135 = 23.94.

Divide each: 20.09 ÷ 23.94 = **0.839.** 2.718 ÷ 23.94 = **0.114.** 1.000 ÷ 23.94 = **0.042.** 0.135 ÷ 23.94 = **0.006.**

Sum: 1.001. Valid distribution, all positive, correctly ordered. That is softmax, and it is two steps: make positive, then express as a share of the total.

Solution (d). The naive alternative: add 2 to everything, giving 5.0, 3.0, 2.0, 0.0. Total 10. Divide: 0.50, 0.30, 0.20, 0.00.

Technically valid — all in [0, 1], sums to 1. So why is it not used? Two reasons, and both matter.

First, it is not decisive. Compare the top scores: softmax gave 0.839, shift-and-divide gave 0.50. The gap between the winner and the runner-up was 2.0 in raw score, and softmax turned that into roughly 7-to-1 while shifting turned it into 1.7-to-1. A model that shows 50% confidence in its clearly best answer produces mushy, indecisive output. Exponentiating amplifies gaps, which is exactly what you want from something whose job is to choose.

Second, the last token got exactly zero — permanently excluded. That is bad practically, because a token with zero probability contributes zero gradient during training, so the model can never learn to raise it. Softmax gave it 0.006: small, alive, trainable.

And a third, quieter problem: the amount you shift by is arbitrary. Shift by 2 and you get one distribution; shift by 100 and you get almost a uniform one. A method whose answer depends on an arbitrary constant is not a method. Softmax has no free parameter.

Common mistake. Concluding that exponentiating is a mathematical necessity — that nothing else *could* work. Part (d) shows an alternative that works and is simply worse. The right lesson is that softmax is a design choice with two good properties (guaranteed positivity, amplified gaps) and no arbitrary constants, not that it is the only option.

8.4 A scaling chart in a vendor deck

A · M · G

Problem. No arithmetic beyond reading. A vendor shows a chart: x-axis is training compute, labelled 10^{-1}, 10^{0}, 10^{1}, 10^{2}, 10^{3} at even spacing. Y-axis is model loss, labelled 4.0, 2.0, 1.0, 0.5, 0.25 at even spacing. The data forms a straight line falling from top-left to bottom-right. (a) What kind of axes are these? (b) What does a straight line mean here? (c) The vendor says "loss keeps improving steadily with scale — the trend is linear." Is that a fair description? (d) You have budget to increase compute tenfold. Reading the chart, what improvement should you expect, and what would you ask the vendor?

Solution (a). Both axes are logarithmic. The tell is that evenly spaced labels represent constant *multiplication* rather than constant addition — each x-step multiplies compute by 10, and each y-step halves the loss. On a linear axis, even spacing means even differences; here it means even ratios.

Students should get into the habit of reading axis labels before the line. Two log axes and a straight line is an extremely common chart in this field and it is routinely misread.

Solution (b). A straight line on log-log axes means a *power law*: multiply compute by a constant factor and loss is multiplied by a constant factor. It does not mean equal amounts of improvement for equal amounts of compute.

Read the specific rate off this chart. One x-step is ×10 compute; one y-step is ×0.5 loss. So on these axes, **ten times the compute halves the loss.** That is the whole content of the line, and it is a sentence anyone in the room can act on.

Solution (c). No — "linear" is doing misleading work. The line is straight on the chart, so the word is not a lie, but the impression it creates is wrong.

"Linear" invites the reader to think each additional unit of compute buys an additional unit of improvement. What is actually happening is that each additional *tenfold* increase buys a halving. The first tenfold takes loss from 4.0 to 2.0 — a gain of 2.0. The next tenfold takes it from 2.0 to 1.0 — a gain of 1.0. Ten times the spend for half the benefit of the previous step.

That is diminishing returns, and it is the single most important economic fact about scaling. The log axes have made it look like steady progress, which is precisely why the chart is drawn this way.

Solution (d). Tenfold compute is one x-step, so one y-step: **loss halves.** If you are at 1.0 you should expect 0.5.

Now the three questions to ask the vendor. First: does loss translate into anything my users notice? A halving of loss may be a large or an invisible change in output quality, and loss is not a product metric. Second: where does the line stop being straight? Power laws hold over a range and then flatten — if the flattening starts just past the right edge of this chart, my tenfold spend buys nothing, and the chart has been cropped to hide that. Third: what were the axis units — is 10^{0} their smallest experiment or my current position?

The general habit: on any log-scale chart, convert the visual claim into a sentence with a multiplication in it before you believe anything. "Ten times the compute halves the loss" is checkable and budgetable. "The trend is linear" is neither.

Common mistake. Extrapolating a log-log straight line off the right-hand edge of the chart. The line is straight over the range that was measured; nothing about a power law guarantees it continues. Every scaling curve eventually bends, and vendors crop their x-axis at the last point where it had not yet bent.

TOPIC 09 · SIX PROBLEMS

Gradients & learning

Nobody will differentiate by hand in their job. Everybody will read a loss curve and choose a learning rate.

How many problems earn competence here: six. Two on the update loop by hand, because a student who has never turned the crank does not believe training is this simple. One on the learning rate, which is the parameter they will actually touch. Two on diagnosing curves, which is the skill they will use weekly. And one judgement problem, because "should we keep training?" is a question about money, not mathematics. The calculus itself gets no problems at all — it is automated, and drilling it would be teaching a skill nobody uses.

9.1 Three steps of gradient descent, by hand

M

Problem. A one-weight model predicts $\hat{y} = w \cdot x$. We have one example: $x = 2$, true $y = 10$. Loss is squared error, $L = (\hat{y} - y)^2$. The gradient of this loss with respect to w is $2(\hat{y} - y) \cdot x$. Start at $w = 1$ with learning rate $\alpha = 0.05$. Perform three update steps, showing prediction, loss and gradient at each.

Solution. The loop is always the same four moves: predict, measure the loss, compute the gradient, step against it. Turn the crank three times.

Step 1. $w = 1$. Predict: $1 \times 2 = 2$. We wanted 10, so we are badly under-predicting.

Loss: $(2 - 10)^2 = (-8)^2 = 64$.

Gradient: $2 \times (2 - 10) \times 2 = 2 \times (-8) \times 2 = -32$.

Update: $w = 1 - 0.05 \times (-32) = 1 + 1.6 =$ **2.6.**

Pause on that minus sign, because it is the whole idea. The gradient is −32, meaning "increasing w decreases the loss". Subtracting a negative adds. So w went up — which is what we wanted, since we were under-predicting. The minus sign in the update rule is what converts "direction of increasing error" into "direction to move".

Step 2. $w = 2.6$. Predict: $2.6 \times 2 = 5.2$. Better — was 2, now 5.2, target 10.

Loss: $(5.2 - 10)^2 = (-4.8)^2 = 23.04$. Down from 64.

Gradient: $2 \times (-4.8) \times 2 = -19.2$. Still negative, so keep increasing w — but smaller in magnitude than −32, because we are closer.

Update: $w = 2.6 + 0.05 \times 19.2 = 2.6 + 0.96 =$ **3.56.**

Step 3. $w = 3.56$. Predict: 7.12. Loss: $(7.12 - 10)^2 = (-2.88)^2 = 8.29$.

Gradient: $2 \times (-2.88) \times 2 = -11.52$. Update: $w = 3.56 + 0.576 =$ **4.136.**

Track the loss across the three steps: 64 → 23.04 → 8.29. And the step sizes: 1.6, then 0.96, then 0.576.

The steps are shrinking automatically, and nobody made them shrink. As the error falls, the gradient falls, so the step falls. Gradient descent naturally decelerates as it approaches the answer — it slams the brakes without being told. That is worth pointing out explicitly, because students expect it to overshoot and oscillate.

The correct answer here is $w = 5$, since $5 \times 2 = 10$. We are at 4.136 after three steps and closing. Notice we never solved for 5 — we crawled towards it, using only local slope information, exactly as in the foggy-hill analogy.

Common mistake. Sign errors — either dropping the minus in the update rule, or computing the error as $(y - \hat{y})$ while using a gradient formula written for $(\hat{y} - y)$. Both send the weight the wrong way and the loss climbs. Diagnostic: if loss increases after a step, check the sign before you touch the learning rate. Nine times out of ten it is a sign, not a rate.

9.2 The same model, three learning rates

M

Problem. Same setup as 9.1 — x = 2, y = 10, w starting at 1, gradient $2(\hat{y} - y) \cdot x$. Take two steps with each of α = 0.01, α = 0.05 and α = 0.30. Report w and loss after each. What happens in the third case, and why?

Solution. Gradient at w = 1 is −32 in all three cases — the gradient does not know or care what the learning rate is. Only the step length differs.

α = 0.01. Step 1: w = 1 + 0.32 = 1.32. Predict 2.64, loss $(2.64 - 10)^2 = 54.17$.

Gradient now 2 × (−7.36) × 2 = −29.44. Step 2: w = 1.32 + 0.294 = 1.614. Predict 3.23, loss 45.83.

Loss went 64 → 54.17 → 45.83. Falling, correctly, and very slowly. At this rate it needs dozens of steps. This is the "learning rate too small" signature: healthy direction, unaffordable pace.

α = 0.05. From 9.1: w = 2.6 then 3.56, loss 64 → 23.04 → 8.29. Fast, stable, decelerating. This is what healthy training looks like.

α = 0.30. Step 1: w = 1 − 0.30 × (−32) = 1 + 9.6 = 10.6.

Predict: 10.6 × 2 = 21.2. Loss: $(21.2 - 10)^2 = 125.44$.

Loss went *up*, from 64 to 125.44. We were under-predicting by 8 and now we are over-predicting by 11.2. We leapt clean over the valley.

Gradient now: 2 × (+11.2) × 2 = +44.8 — positive, so it correctly tells us to reduce w. Step 2: w = 10.6 − 0.30 × 44.8 = 10.6 − 13.44 = −2.84.

Predict: −5.68. Loss: $(-5.68 - 10)^2 = 245.86$.

Loss: 64 → 125.44 → 245.86. It is **diverging**, roughly doubling each step, and w is oscillating with growing amplitude: 1, 10.6, −2.84.

And here is the crucial diagnostic point: the gradient was correct every single time. It always pointed the right way. The failure is entirely in the step length — we travelled so far along a correct direction that we arrived somewhere the direction no longer applied. A gradient is local information, and a large step spends it in territory where it was never valid.

Continue this for a few more steps and the numbers exceed float range: that is the NaN in a real training log. Not a mysterious bug — this, compounding.

Common mistake. Diagnosing the α = 0.30 case as "the gradient is wrong" or "the model is broken". Both are intact. Also common: fixing divergence by reducing α by ten percent. The useful range spans orders of magnitude, so adjust by dividing by 3 or 10 — a ten percent change is searching a city one street at a time.

9.3 Five loss curves, five diagnoses

M

Problem. Diagnose each and state the action. (a) Training loss falls smoothly then flattens at 0.4; validation loss tracks it and also flattens at 0.42. (b) Training loss falls to 0.05; validation loss falls to 0.6 then rises to 0.9. (c) Training loss oscillates between 2.1 and 3.8 with no trend over 500 steps. (d) Training loss falls from 4.0 to 3.7 over 2,000 steps, still falling. (e) Training loss falls normally for 300 steps then becomes NaN.

Solution (a). Healthy, converged. Action: stop.

Two things make this healthy, and both need checking. The curve flattened, so further steps buy nothing. And validation tracks training closely — 0.40 against 0.42 — so the model learned the pattern rather than memorising the examples. Continuing to train costs money and adds nothing.

If you want a lower loss from here, more steps is the wrong lever. You need more data, a bigger model, or better features.

Solution (b). Overfitting. Action: stop at the validation minimum and regularise.

The signature is the divergence: training keeps improving while validation turns around. The model is memorising the training set — it is getting better at the exam and worse at the subject.

And note the trap: if you plotted only the training curve you would see 0.05 and celebrate. The gap is the diagnosis, and a gap is invisible when you plot one line. Always plot both on the same axes.

Actions: revert to the checkpoint at validation's minimum of 0.6; then add dropout or weight decay, get more training data, or reduce model size. The rise from 0.6 to 0.9 is pure waste — the model spent compute making itself worse.

Solution (c). Learning rate too high. Action: divide it by 3 or 10.

This is problem 9.2's third case, caught before it diverged. Oscillation with no trend means each step overshoots and the next overshoots back — bouncing between the valley walls, never descending. Note the loss is not increasing, so it is not the runaway case; the steps are large enough to prevent progress but not large enough to explode.

Also consider a warmup schedule: start with a very small rate for the first few hundred steps, then raise it. Early in training the weights are random and gradients are wild, and warmup stops those first violent steps from throwing the model somewhere unrecoverable.

Solution (d). Learning rate too low. Action: increase it by 3× or 10×.

The direction is right — loss is genuinely falling — but 0.3 of progress in 2,000 steps means the run finishes some time next month. This is problem 9.2's first case.

One caution to teach alongside: a very low learning rate can look like a converged curve if you only glance at it, because both are nearly flat. The distinction is whether it is *still falling*. Check the slope over the last few hundred steps rather than eyeballing the shape.

Solution (e). Overflow — the step exploded. Action: clip gradients, reduce the rate, and check your data.

NaN means a number left the representable range and every subsequent calculation involving it became meaningless. Usually a large gradient produced a large step, which produced absurd weights, which produced an even larger gradient — the runaway loop from 9.2, compounding until it exceeded float range.

Gradient clipping is the direct fix: cap the step length regardless of what the gradient asks for. But 300 successful steps before failure is a clue worth following — a sudden explosion after stable training often means a specific bad batch. Check for extreme values, division by near-zero, or a corrupted record around step 300. Fixing the data is better than clipping around it.

Give the class the reading order for any curve: is it falling? is it still falling? does validation track it? That is three questions and it covers all five cases.

Common mistake. Diagnosing (d) as converged and shipping the model. It is not converged, it is crawling — and the difference is worth a great deal of accuracy. Also frequent: responding to (b) by training longer in the hope that validation loss will come back down. It will not; the divergence is monotonic once it starts.

9.4 Forecast error and which way the weights move

A · M

Problem. A demand-forecasting model predicts weekly units for four SKUs. Predictions: 220, 180, 95, 310. Actuals: 180, 195, 95, 260. (a) Compute each error as actual minus predicted. (b) For each, state whether the model over- or under-predicted and which way its weights should move. (c) Compute the mean error and the mean absolute error. Why do they differ, and which should you report?

Solution (a). Error = actual – predicted.

SKU 1: 180 – 220 = **–40.** SKU 2: 195 – 180 = **+15.** SKU 3: 95 – 95 = **0.** SKU 4: 260 – 310 = **–50.**

Solution (b). The sign is the instruction.

SKU 1, error –40: negative means actual was below predicted, so the model over-predicted. Weights driving this prediction should come *down*.

SKU 2, error +15: under-predicted. Weights should go *up*.

SKU 3, error 0: perfect. Gradient is zero, so this example produces no update at all. Worth stating — an example the model already gets right contributes nothing to learning, which is why datasets of easy examples train slowly.

SKU 4, error –50: over-predicted, weights down, and this is the largest error so it will pull hardest.

Now the important observation: these four examples disagree. Three want the weights lower, one wants them higher. The update is a negotiation — the weights move in whatever direction reduces total loss, and SKU 2's prediction will get *worse* as a result. The model is not trying to be right about any individual SKU. It is trying to be least wrong overall.

Solution (c). Mean error: (–40 + 15 + 0 – 50) ÷ 4 = –75 ÷ 4 = **–18.75.**

Mean absolute error: (40 + 15 + 0 + 50) ÷ 4 = 105 ÷ 4 = **26.25.**

They differ because the mean error lets over- and under-predictions cancel. Here they partially cancel — the +15 offsets some of the –90 — so the mean error understates how wrong the model is.

Which to report? **Both, and they answer different questions.** Mean error measures *bias*: at –18.75 this model systematically over-forecasts, which for inventory means consistent overstocking. That is a fixable, directional problem. Mean absolute error measures *magnitude*: typical miss of 26 units, regardless of direction.

And the failure mode to warn about: a model could have errors of +100 and –100, giving a mean error of exactly zero. Reported alone, that looks like a perfect forecast. It is a wildly inaccurate model that happens to be unbiased. Never report mean error without a magnitude measure beside it.

Common mistake. Defining error as predicted minus actual in one place and actual minus predicted in another. Both conventions are fine; mixing them within a single piece of work flips signs and sends weights the wrong way. Pick one, write it at the top of the file, and never deviate.

9.5 Why a fine-tune uses a smaller learning rate

M · G

Problem. No arithmetic. A team fine-tunes a general-purpose model on 4,000 internal support conversations, reusing the learning rate from a from-scratch training run. After training, the model answers support questions in the right format but has become noticeably worse at general reasoning and occasionally produces confidently wrong facts it previously got right. Explain what happened and prescribe the fix.

Solution. This is catastrophic forgetting, and it is entirely explainable with the foggy-hill picture.

What the starting point is. A pre-trained model is not at a random position in the landscape. It is already at the bottom of a good valley — its weights encode general language, facts and reasoning, found over months of training on a trillion tokens. Fine-tuning starts there.

What a from-scratch learning rate is for. At the start of training from random weights, you are on a mountainside and you need to cover ground. Large steps are appropriate — there is nothing valuable to protect and a great distance to travel.

What goes wrong. Take mountain-sized steps from the bottom of a good valley and you climb straight out of it. The gradients from 4,000 support conversations point towards a narrow valley that is excellent for support formatting and knows nothing about general reasoning — because the fine-tuning data contains no general reasoning to preserve.

The loss on the fine-tuning task falls beautifully, so the training log looks like a success. Everything the fine-tuning data does not measure quietly degrades, and nothing in the run reports it.

Notice also the data-size mismatch: 4,000 conversations against a trillion pre-training tokens. You are letting a very small sample overwrite the conclusions of a very large one — and with a large learning rate, the small sample wins.

The fix, in four parts.

First, reduce the learning rate substantially — commonly by a factor of ten to a hundred. You are nudging, not building. Small steps keep the model inside the valley it already occupies while adjusting its position within it.

Second, train for far fewer steps. With 4,000 examples, a handful of passes is usually enough; twenty passes is memorisation.

Third — and this is the part teams skip — evaluate on tasks the fine-tune is *not* about. Keep a small general-capability set and run it before and after. This is the only way to see the damage, because the fine-tuning loss curve will never show it.

Fourth, consider parameter-efficient methods such as LoRA, which train a small set of additional weights and leave the original ones untouched. Forgetting is structurally limited because the pre-trained weights cannot move.

The general principle worth stating: **the right learning rate depends on how good your starting point is.** Random weights want large steps. A well-trained model wants small ones. Reusing a rate across those two situations is reusing a number whose meaning changed.

Common mistake. Diagnosing the confident wrong facts as a hallucination problem and reaching for prompt engineering or RAG. The cause is upstream: the fine-tune damaged the weights that held those facts. No prompt recovers information that has been overwritten. Roll back and retrain properly.

9.6 Should we keep training?

A · M · G

Problem. A run has been going for six days. Validation loss: day 1, 2.40. Day 2, 1.62. Day 3, 1.31. Day 4, 1.19. Day 5, 1.14. Day 6, 1.12. Compute costs ₹38,000 per day. (a) Compute the daily improvement. (b) What does the pattern tell you? (c) Someone proposes six more days to "get under 1.0". Evaluate that. (d) What would you actually do?

Solution (a). Day-on-day reductions in validation loss:

Day 1→2: 0.78. Day 2→3: 0.31. Day 3→4: 0.12. Day 4→5: 0.05. Day 5→6: **0.02.**

Solution (b). Each day delivers roughly a third to a half of the previous day's gain. That is not a slowdown you can wait out — it is the shape of the whole process. The curve is flattening geometrically.

Put the cost against it. Day 2 bought 0.78 of loss reduction for ₹38,000, about ₹49,000 per unit of loss. Day 6 bought 0.02 for the same ₹38,000 — about ₹1.9 million per unit. **Day 6 was roughly forty times more expensive per unit of improvement than day 2.** Same spend, same hardware, forty-fold worse value.

Solution (c). It will not work, and the arithmetic says so.

Extrapolate the pattern. If each day yields about 40% of the previous day's gain, then day 7 gives roughly 0.008, day 8 about 0.003, and so on. Sum the remaining gains and the series converges to about 0.013 — the loss asymptotes near 1.107.

Six more days at ₹38,000 is ₹228,000, and it buys perhaps 0.01 of loss. It does not reach 1.0. It does not reach 1.10. The target was chosen because it is a round number, not because anything about the curve suggests it is attainable.

And note this is the same lesson as the log-scale chart in 8.4: improvement is multiplicative, so equal amounts of spend buy geometrically less. Anyone who reads the curve as "steadily improving, keep going" has read a linear story into a geometric one.

Solution (d). Stop the run — and then ask the question nobody has asked.

Stop, because the marginal day is now buying 0.02 of a metric for ₹38,000. Take the day-6 checkpoint.

Then the missing question: **does a validation loss of 1.12 versus 1.10 change anything a user experiences?** Loss is a training signal, not a product metric. Somewhere between "1.12" and "the assistant is useful" there is a translation nobody in this conversation has performed. If a 0.02 improvement in loss moves no task metric that anyone cares about, the entire debate about six more days was about nothing.

And where to spend the ₹228,000 instead: more or better training data, which shifts the asymptote rather than crawling towards it; a larger model, same reason; or a proper evaluation set, per problem 3.5, so that next time you can tell whether an improvement is real. All three change the ceiling. More days of the same run only approaches the existing one.

Common mistake. Setting the target first — "we need loss under 1.0" — and then asking how long it takes. The curve determines what is reachable; a target chosen independently of it is a wish. Make students extrapolate the series before they accept any loss target, and the impossible ones announce themselves in one line of arithmetic.

TOPIC 10 · FIVE PROBLEMS

Softmax & token prediction

The dial everyone turns and few can explain. Getting temperature right is worth more than most prompt engineering.

How many problems earn competence here: five. One full softmax by hand — non-negotiable, because students who have not computed one treat it as a black box. Two on temperature, since it is the parameter they will actually set and the mechanism is genuinely counter-intuitive. One on reading a distribution as a product signal. And one judgement problem on temperature zero, which is the most widely misunderstood setting in applied GenAI.

10.1 Softmax over five intents

M · G

Problem. An intent classifier produces logits for five intents: refund 2.5, track_order 1.5, cancel 0.5, complaint −0.5, greeting −1.5. (a) Convert to probabilities. (b) Verify they sum to 1. (c) The logit gap between refund and track_order is 1.0 — what probability ratio did that become, and why? (d) If your routing rule is "act automatically above 0.70, else ask a clarifying question", what happens here?

Solution (a). Two steps: exponentiate, then divide by the total.

$e^{2.5} \approx 12.18$. $e^{1.5} \approx 4.48$. $e^{0.5} \approx 1.65$. $e^{-0.5} \approx 0.61$. $e^{-1.5} \approx 0.22$.

Note that the two negative logits produced positive numbers — 0.61 and 0.22 — as they must. And the ordering is untouched.

Sum: 12.18 + 4.48 + 1.65 + 0.61 + 0.22 = 19.14.

Divide each by 19.14: refund **0.636**, track_order **0.234**, cancel **0.086**, complaint **0.032**, greeting **0.011**.

Solution (b). 0.636 + 0.234 + 0.086 + 0.032 + 0.011 = 0.999. One, within rounding. Always run this check — it is free and it catches arithmetic slips immediately.

Solution (c). 0.636 ÷ 0.234 = **2.72 to 1** — and that number should look familiar. It is e, because a logit gap of exactly 1.0 becomes a probability ratio of exactly e^1.

That is worth teaching as a reflex: **each 1.0 of logit gap is a factor of e, about 2.7.** A gap of 2.0 is e^2, about 7.4 to one. A gap of 3.0 is about 20 to one. With that one fact you can read logits at a glance without exponentiating anything.

And notice the amplification: the raw scores 2.5 and 1.5 are not dramatically different — if you simply expressed them as shares of the raw total you would get something like 1.6 to 1. Exponentiating turned a modest lead into a decisive one, which is precisely what you want from a mechanism whose job is to choose.

Solution (d). The top probability is 0.636, below the 0.70 threshold. So the system asks a clarifying question rather than acting.

And that is the right call, visible in the numbers. Refund leads, but track_order holds 0.234 — nearly a quarter of the probability mass. Automatically initiating a refund when there is a one-in-four chance the customer wanted to track a parcel is an expensive mistake made confidently.

Worth pointing out to the class: the threshold did work here, and only because someone set one. Without it, the system would have taken the argmax and acted on 0.636 as though it were certainty. The distribution contained the doubt; the threshold is what let the system act on it.

Common mistake. Exponentiating and then normalising against the wrong total — for instance, dividing by the sum of the original logits (2.5 + 1.5 + 0.5 − 0.5 − 1.5 = 2.5) instead of the sum of the exponentials. The result exceeds 1 and does not sum correctly. Insist on the sum-to-one check every time; it catches this instantly.

10.2 The same logits at three temperatures

G

Problem. Three logits: 2.0, 1.0, 0.0. Compute the probability distribution at (a) $\tau = 1.0$, (b) $\tau = 0.5$, (c) $\tau = 2.0$. (d) Describe the mechanism in one sentence, and state which setting suits extraction and which suits brainstorming.

Solution. Temperature divides every logit before softmax sees it. That is the entire operation — divide, then run softmax as normal.

(a) $\tau = 1.0$. Dividing by 1 changes nothing. Logits stay 2.0, 1.0, 0.0.

$e^2 \approx 7.39$, $e^1 \approx 2.72$, $e^0 = 1.00$. Sum 11.11.

Probabilities: **0.665, 0.245, 0.090.**

(b) $\tau = 0.5$. Divide each logit by 0.5 — which doubles them. New logits: 4.0, 2.0, 0.0.

Look at what happened to the gaps first: they were 1.0 apart and are now 2.0 apart. Dividing by a number below one *multiplies* the gaps.

$e^4 \approx 54.60$, $e^2 \approx 7.39$, $e^0 = 1.00$. Sum 62.99.

Probabilities: **0.867, 0.117, 0.016.** Sharper — the leader gained 20 points.

(c) $\tau = 2.0$. Divide by 2: logits become 1.0, 0.5, 0.0. Gaps halved to 0.5.

$e^1 \approx 2.72$, $e^{0.5} \approx 1.65$, $e^0 = 1.00$. Sum 5.37.

Probabilities: **0.506, 0.307, 0.186.** Flatter — the leader dropped to barely half, and the third option went from 9% to 19%.

Solution (d). The mechanism in one sentence: **temperature rescales the gaps between logits, and softmax converts gaps into decisiveness — so dividing by a small number widens gaps and sharpens the distribution, while dividing by a large number narrows gaps and flattens it.**

Extraction wants τ near 0. Pulling an invoice number out of a document has exactly one right answer, so variety has no value and every point of randomness is a chance to corrupt the output. Look at the third option: 9% at $\tau = 1$, 1.6% at $\tau = 0.5$. At $\tau = 0.2$ it would be effectively zero.

Brainstorming wants τ above 1. If you ask for ten campaign ideas at $\tau = 0.5$, the model will produce ten variations on its single favourite. At $\tau = 1.5$ the third and fourth candidates get real probability, and you get genuine variety — some of it bad, which is the point of brainstorming.

Common mistake. Believing temperature multiplies the probabilities rather than dividing the logits. It operates *before* exponentiation, which is why its effect is non-linear — halving τ does not halve anything, it squares the ratios. Students who think it scales probabilities cannot predict what $\tau = 0.5$ will do.

10.3 Choosing temperature for six real tasks

G

Problem. Choose a temperature and justify it in one sentence. (a) Extracting GSTIN numbers from scanned invoices into JSON. (b) Writing three subject-line options for a marketing email. (c) Classifying support tickets into eight categories. (d) Summarising a 40-page contract for a lawyer. (e) Generating synthetic training examples for a classifier. (f) A code assistant completing a function signature.

Solution. One question decides every case: *how many acceptable answers are there?* Exactly one means near zero. Many means higher.

(a) $\tau \approx 0$. A GSTIN is a specific string; there is precisely one correct output. Randomness cannot improve it and can only corrupt a character. And note the failure mode: a single wrong digit produces a plausible-looking invalid number that passes visual inspection and fails downstream.

(b) $\tau \approx 1.0$ to 1.3. You explicitly asked for three *different* options, and low temperature would give you three rephrasings of one idea. The whole value of this task is spread.

(c) $\tau \approx 0$. Classification into a fixed set has one right label per ticket. Same reasoning as (a) — and additionally, you want the same ticket to get the same category every time, or your reporting is unreliable.

(d) $\tau \approx 0.2$ to 0.4, and this is the interesting one. Students usually say "summarising is creative, so higher". It is not. There is no single correct summary, so $\tau = 0$ is unnecessary — but every clause in a legal summary must be faithful to the source, and higher temperature is precisely where invented specifics come from. Low, but not zero.

(e) $\tau \approx 1.2$ to 1.5, higher than most people expect. The purpose of synthetic training data is coverage — you want unusual phrasings, edge cases, awkward constructions. Low temperature would generate a thousand near-identical examples, which teaches the classifier almost nothing and inflates your dataset with duplicates. This is the case where high temperature is not a stylistic preference but the entire point.

(f) $\tau \approx 0.1$ to 0.2. A function signature in an existing codebase has essentially one right form — it must match the conventions and the call sites already written. Low. Note the contrast with generating a whole new function from a description, where τ around 0.5 is reasonable because several implementations are genuinely valid.

The pattern worth stating: temperature is not a creativity dial, it is a **tolerance-for-alternatives** dial. Ask how many outputs would satisfy you. One — go to zero. Several equally good — go up. And notice (d) and (f) show the answer is often "low but not zero", which is the setting people never reach for because the framing as "creative versus factual" hides it.

Common mistake. Sorting tasks by how "creative" they sound rather than by how many acceptable answers exist. Summarisation sounds creative and needs low temperature; generating synthetic data sounds mechanical and needs high. The intuitive axis is the wrong axis.

10.4 A distribution as a product signal

A · G

Problem. A diagnostic assistant returns, for one patient: viral_infection 0.41, bacterial_infection 0.33, allergic_reaction 0.19, other 0.07. For a second patient: viral_infection 0.94, bacterial_infection 0.03, allergic_reaction 0.02, other 0.01. Both top answers are the same label. (a) What does each distribution say? (b) Should the interface treat them identically? (c) Design the rule.

Solution (a). Patient one: the model is genuinely torn. Viral leads at 0.41, but bacterial at 0.33 is close behind, and together the two infection hypotheses hold 0.74 while allergic reaction retains a real 0.19. In plain terms the model is saying "probably an infection, and I cannot tell you which kind".

Patient two: near-certainty. 0.94 on one label, with everything else in the noise. The model has seen this pattern many times and it points one way.

Solution (b). No — and this is the whole point.

If the interface displays only the top label, both patients produce the identical screen: "Viral infection". A single confident-looking phrase. The clinician has no way to know that one of those was a 41% guess between two clinically different conditions with different treatments.

The information existed. The model produced it correctly. The interface threw it away. That is not a machine learning failure — it is a design failure, and it is the most common one in applied AI.

Solution (c). A workable rule has three bands and uses two numbers, not one.

Above 0.85, and the gap to second place above 0.5: present the single answer, with confidence shown. Patient two qualifies.

Top between 0.5 and 0.85, or a narrow gap: present the top two or three with their probabilities, explicitly framed as alternatives to rule out. Patient one lands here, and the clinician sees "viral 41%, bacterial 33%" — which is genuinely useful information rather than a wrong single answer.

Top below 0.5: present as "insufficient signal", list the candidates, and recommend a specific test that would discriminate between them.

Note why the *gap* matters as well as the top value. A distribution of 0.86 / 0.12 is confident. A distribution of 0.86 / 0.13 with the rest at zero is also confident — but 0.51 / 0.49 and 0.51 / 0.10 are very different situations despite sharing a top value. Threshold on both.

And a closing note for a high-stakes domain: this rule also gives you an auditable escalation policy. When a decision is later questioned, "the model reported 0.41 and the interface showed alternatives" is a defensible position. "The model said viral" is not.

Common mistake. Trying to fix patient one by lowering the temperature so the distribution sharpens. Temperature would push 0.41 towards 0.7 or higher — but the model's underlying uncertainty is unchanged. You would have manufactured false confidence and destroyed the only signal that told you to be careful. Never sharpen a distribution to make a threshold pass.

10.5 "We set temperature to zero, so it can't be wrong"

A · M · G

Problem. No arithmetic. A team reports: "We were getting occasional wrong answers from our document assistant. We set temperature to 0 and the problem is solved — it now gives the same answer every time." Evaluate this claim precisely.

Solution. Something real did improve, and the stated conclusion is wrong. Both halves matter.

What temperature actually touches. It rescales the gaps between logits the model has already produced. It cannot add knowledge, correct a fact, or change which token ranked first. At $\tau = 0$ you always take the argmax — the token the model already considered most likely.

So if the model's top-ranked answer is wrong, $\tau = 0$ guarantees you receive that wrong answer *every single time*, with perfect reliability. The error rate did not fall. It became deterministic.

Why it felt like a fix. Two reasons, and they are worth separating.

First, variance in the output looks like unreliability to a user. Asking the same question three times and getting three phrasings feels broken even when all three are correct. Removing the variance removes that feeling, which is a genuine improvement in perceived quality.

Second — the real effect — at higher temperature the model sometimes samples its second or third choice, and second choices are wrong more often than first choices. So $\tau = 0$ does eliminate that specific class of error: the occasions when sampling picked a worse token than the model's own best guess. That is a real reduction in errors.

What it cannot touch is the case where the model's *first* choice was wrong. And in a document assistant, that is usually the dominant failure — the retrieved context was irrelevant, or the model was confidently mistaken. Those errors are entirely unaffected.

The precise statement to replace theirs: "Temperature 0 made our outputs deterministic and removed errors caused by sampling away from the model's top choice. It did not affect errors where the top choice itself was wrong, and we have not yet measured how much of our error was which."

That last clause is the actionable one. The team should measure. Run the same evaluation set at $\tau = 0$ and at their previous setting, and compare error rates. If the error rate fell by 2 points, sampling was a minor contributor and the real problem is upstream — retrieval, per problem 5.5.

One genuine benefit worth crediting them for: determinism makes the system *testable*. You can write a regression suite with fixed expected outputs, which is impossible at higher temperature. Consistency is a real engineering good. It is simply not the same property as correctness.

Common mistake. Overcorrecting — telling the team that temperature 0 achieved nothing. It reduced a real error class and bought testability. The error is in the word "solved", and in stopping the investigation. A student who dismisses the change entirely has learned the slogan rather than the mechanism.

TOPIC 11 · FOUR PROBLEMS

Information theory

Entropy is average surprise. That one sentence, properly understood, covers cross-entropy loss, perplexity and drift monitoring.

How many problems earn competence here: four. One entropy calculation by hand, because the formula's minus sign and probability weighting both need to be felt. One on cross-entropy as a training signal, since it is the loss function behind every language model. One on perplexity, where the comparability trap lives. And one on entropy as a live monitoring signal, which is the use most likely to earn its keep in production. There is no fifth distinct use in this session.

11.1 Entropy of three routing distributions

M

Problem. A ticket router outputs probabilities over four queues. Case A: [0.97, 0.01, 0.01, 0.01]. Case B: [0.50, 0.30, 0.15, 0.05]. Case C: [0.25, 0.25, 0.25, 0.25]. Compute the entropy of each in bits, using $\log_2(0.97) \approx -0.044$, $\log_2(0.5) = -1$, $\log_2(0.3) \approx -1.737$, $\log_2(0.25) = -2$, $\log_2(0.15) \approx -2.737$, $\log_2(0.05) \approx -4.322$, $\log_2(0.01) \approx -6.644$.

Solution. Entropy is $H = -\Sigma\, p_i \log_2 p_i$. Three moves per term: take the log, multiply by the probability, then flip the sign at the end.

Case A. Terms: $0.97 \times (-0.044) = -0.0427$. Then three identical terms of $0.01 \times (-6.644) = -0.0664$ each, totalling -0.1993.

Sum: -0.242. Flip the sign: H = **0.242 bits.**

Notice the structure of that answer. The near-certain outcome contributed almost nothing — 0.0427 — because $\log_2(0.97)$ is nearly zero: an event you expected carries almost no information when it happens. Meanwhile the three unlikely outcomes contributed 0.199 between them, over four fifths of the total, despite holding only 3% of the probability. Rare events are individually surprising; the probability weighting is what stops them dominating the average.

Case B. $0.50 \times (-1) = -0.500$. $0.30 \times (-1.737) = -0.521$. $0.15 \times (-2.737) = -0.411$. $0.05 \times (-4.322) = -0.216$.

Sum: -1.648. H = **1.648 bits.**

Note that the 0.30 term contributed slightly *more* than the 0.50 term — 0.521 against 0.500. A moderately likely outcome can carry more of the entropy than the leading one, because it combines a decent probability with a decent surprise. That is the product doing its work.

Case C. Four terms of $0.25 \times (-2) = -0.50$ each. Sum -2.0. H = **2.0 bits.**

Two bits is the maximum possible for four outcomes, and there is a clean reason: with four equally likely options you need exactly two yes-or-no questions to identify one. This is the router having learned nothing useful — it may as well toss two coins.

So the three cases read 0.24, 1.65, 2.00 bits, against a ceiling of 2.00. Case A is a confident router; Case C is a useless one. And the minus sign at the front of the formula is pure convenience — the logs of probabilities are all negative, and flipping the sign lets us talk about surprise as a positive quantity.

Common mistake. Forgetting to weight by p_i — summing the logs alone. That gives Case A an enormous value driven by three events that almost never happen, and it is no longer an average of anything. Diagnostic question: "should an outcome with probability 0.001 contribute as much to the average as one with probability 0.9?"

11.2 Cross-entropy as the training signal

M

Problem. For a single next-token prediction, the true token is "Mumbai". Three models assign it these probabilities: model X gives 0.80, model Y gives 0.30, model Z gives 0.02. (a) Compute the cross-entropy loss for each, using natural logs: ln(0.8) ≈ −0.223, ln(0.3) ≈ −1.204, ln(0.02) ≈ −3.912. (b) Explain why the loss is defined this way. (c) Which model gets the largest gradient push, and is that correct?

Solution (a). For a single known-correct token, cross-entropy simplifies dramatically: it is just the negative log of the probability assigned to the true token. Every other term is multiplied by zero, because the true distribution puts all its mass on one token.

Model X: −ln(0.80) = **0.223.** Model Y: −ln(0.30) = **1.204.** Model Z: −ln(0.02) = **3.912.**

Solution (b). Read the quantity in words: cross-entropy is *how surprised the model was by what actually happened.*

Model X expected Mumbai and got it — low surprise, low loss, nothing much to fix. Model Z considered Mumbai nearly impossible at 2% and Mumbai arrived — enormous surprise, large loss, and a strong signal that something is badly wrong with its beliefs.

Now why the log rather than, say, one minus the probability? Because of the behaviour at the extremes. With one-minus-p, model Z's loss would be 0.98 and model X's 0.20 — a ratio of about 5. With the log, the ratio is 3.912 to 0.223, about 17.5. And as p approaches zero the log grows without limit, so a model that assigns near-zero probability to something that then happens is punished almost without bound.

That unbounded punishment is deliberate and useful: it makes confident wrongness the most expensive possible error. A model that hedges is treated more kindly than one that is certain and mistaken — which is exactly the incentive you want in a system that will be believed.

Solution (c). Model Z gets the largest push, by a wide margin, and yes — that is correct.

Z's beliefs are furthest from reality, so it has the most to learn from this example. X is nearly right and needs only a nudge. The gradient magnitude tracks the size of the mistake, which means training automatically allocates its effort towards whatever the model is worst at, without anybody scheduling that.

One consequence worth flagging for later: this is also why a single mislabelled training example can do real damage. If the data says "Mumbai" and the truth is "Delhi", the model is confidently correct, cross-entropy punishes it heavily, and it is dragged towards the wrong answer with maximum force. The loss function cannot distinguish a bad model from bad data.

Common mistake. Computing cross-entropy over all four candidate tokens, summing terms for the wrong ones too. For a single known-correct label, only the true token's term survives — every other true-probability is zero, so those products vanish. Students who carry all the terms get an inflated number and lose the direct interpretation as "surprise at the correct answer".

11.3 Two perplexity numbers in a vendor comparison

M · G

Problem. Model P reports a cross-entropy of 2.30 nats on your test set. Model Q reports 3.00. (a) Convert both to perplexity, using $e^{2.30} \approx 10.0$ and $e^{3.00} \approx 20.1$. (b) Interpret each number in plain words. (c) A vendor claims their model has perplexity 8.2, better than both. What must you establish before believing it?

Solution (a). Perplexity is e raised to the cross-entropy. Model P: $e^{2.30}$ = **10.0.** Model Q: $e^{3.00}$ = **20.1.**

Note the compression: a gap of 0.70 in cross-entropy became a factor of two in perplexity. Small-looking differences in loss are large differences in behaviour, which is exactly why perplexity exists as a reporting format.

Solution (b). Perplexity is the effective number of options the model was choosing between at each step.

Model P, at 10.0: on average it had narrowed the next token down to about ten plausible candidates out of a vocabulary of perhaps a hundred thousand. Model Q, at 20.1: about twenty candidates. P is roughly twice as decisive.

And that is the whole reason we exponentiate. "Cross-entropy of 2.30 nats" means nothing to a human. "Choosing between about ten options" is a quantity anyone can picture and reason about.

Solution (c). Three things, and the first is the one everybody skips.

The tokenizer must be the same. Perplexity is per-token, so it depends entirely on what counts as a token. A model that splits text into whole words is predicting a much harder thing at each step than one that splits into short fragments — with fragments, much of the "prediction" is just completing a word it has already started. Different vocabularies mean the two models are being asked different questions, and their scores live on unrelated scales.

This is not a technicality. A model can lower its perplexity substantially by adopting a finer tokenizer while getting no better at anything. Comparing perplexity across tokenizers is comparing marks from two different exams.

The test set must be the same. Perplexity is measured on data, and easier data gives lower numbers. A vendor evaluating on general web text will beat your figure measured on domain-specific technical documents, without being better at your work.

And the test data must not have been in their training data. Contamination produces spectacular perplexity on text the model has memorised, and tells you nothing about new inputs.

Then the question that outranks all three: **does perplexity predict anything you care about?** It measures next-token prediction, not helpfulness, accuracy or instruction-following. A model with better perplexity can be worse at your actual task. Ask for task-level evaluation on your own data, and treat perplexity as a diagnostic rather than a decision criterion.

Common mistake. Accepting a cross-tokenizer perplexity comparison because the numbers are presented in the same units. Same units, different meaning — the units say nothing about what was measured. Students should ask "same tokenizer, same test set?" as a reflex before reading any perplexity figure.

11.4 Rising entropy in production

M · G

Problem. Your intent classifier's mean output entropy over the last eight weeks: 0.31, 0.33, 0.30, 0.34, 0.52, 0.61, 0.74, 0.79 bits. Accuracy on your fixed test set has not moved. (a) What is happening? (b) Why did the test set not catch it? (c) What do you do, in what order?

Solution (a). The model is becoming steadily less confident about live traffic. Entropy has more than doubled, with a clear break between week 4 and week 5.

Read what rising entropy means mechanically: the output distributions are flattening. Where the model used to put 0.95 on one intent, it is now spreading 0.6 and 0.3 across two. It is meeting inputs it has no confident answer for.

The step change at week 5 matters. Gradual drift would show a slow ramp; a jump suggests a discrete event — a new product launched, a marketing campaign brought a different customer segment, an upstream system changed how it formats text, or a new channel opened. Something specific happened in week 5, and the entropy series tells you where to look in the changelog.

Solution (b). Because the test set is fixed, and the world is not.

Your test set was assembled some time ago from the traffic distribution as it was then. The model is still exactly as good at that distribution — accuracy correctly reports no change. What changed is that live traffic no longer resembles the test set.

This is problem 3.6 arriving from the other direction. There, a large biased sample gave a precise answer to the wrong question. Here, a valid test set has quietly become the wrong question because time passed. Both are sampling-frame failures rather than measurement failures.

And note why entropy caught what accuracy could not: **entropy needs no labels.** It is computed from the model's own output on live traffic, so it works on the data you actually have, in real time. Accuracy requires ground truth, which means waiting for labels — usually weeks — on data somebody has to annotate. Entropy is the early-warning system precisely because it is unsupervised.

Solution (c). Four steps, in this order.

First, sample the high-entropy inputs and read them. Pull a hundred of the flattest-distribution cases from weeks 7 and 8 and look at them with your own eyes. This costs an afternoon and usually identifies the cause outright — a new product name, a different language, a new phrasing convention.

Second, check what changed in week 5. Deployments, upstream format changes, campaigns, new channels. Correlate the break with the changelog.

Third, route high-entropy cases to humans now, before you fix anything. This is the immediate mitigation: entropy gives you a per-request confidence signal, so you can escalate exactly the requests the model is unsure about rather than degrading service for everyone.

Fourth, refresh the test set and then retrain. In that order — retraining against a stale test set means you cannot tell whether you fixed anything. And make the refresh recurring, because the lesson of this problem is that a fixed test set has a shelf life.

Common mistake. Retraining immediately on recent data without looking at the high-entropy inputs first. You may fix the symptom without ever learning what happened — and if the cause was an upstream formatting bug rather than genuine drift, you will have trained your model to accommodate corrupted input. Diagnose before you retrain.

TOPIC 12 · THREE PROBLEMS

Numerical stability

One idea: formulas that are equivalent in mathematics are not equivalent in floating point.

How many problems earn competence here: three, and that is the honest ceiling. One idea and one trap. One problem on the max-subtraction trick, one on precision formats and memory, one judgement problem on when to trust a library over your own implementation. A fourth would be padding — and padding a short topic teaches students that problem count signals importance, which is the wrong lesson.

12.1 A softmax that returns nonsense

M

Problem. Three logits: 1000, 999, 998. (a) What happens if you exponentiate directly? (b) Subtract the maximum and recompute. (c) Prove the two are mathematically identical. (d) What if the logits were −1000, −1001, −1002?

Solution (a). e^{1000} is not a large number as far as a 32-bit float is concerned. It is infinity — the format tops out around 10^{38}, and e^{1000} is roughly 10^{434}.

So all three exponentials become infinity, and the sum is infinity. The final division is infinity divided by infinity, which is NaN. Every probability comes back as not-a-number, and every downstream calculation is poisoned.

Note how little warning you get: the logits themselves are perfectly ordinary numbers. Nothing about 1000 looks dangerous until it reaches an exponential.

Solution (b). The maximum is 1000. Subtract it from every logit: 0, −1, −2.

$e^0 = 1.000$. $e^{-1} \approx 0.368$. $e^{-2} \approx 0.135$. Sum: 1.503.

Probabilities: 1.000 ÷ 1.503 = **0.665.** 0.368 ÷ 1.503 = **0.245.** 0.135 ÷ 1.503 = **0.090.**

Sensible, sums to one — and note these are exactly the numbers from problem 10.2(a), where the logits were 2, 1, 0. That is not a coincidence: what softmax responds to is the *gaps* between logits, and both sets have gaps of 1. Adding a constant to every logit changes nothing about the answer.

Solution (c). Here is the proof, and it is two lines.

Subtracting m from every logit multiplies every exponential by the same factor, e^{-m}, because e^(z−m) = e^z × e^(−m).

So the numerator is multiplied by e^{-m}, and the denominator — a sum of terms each multiplied by e^{-m} — is also multiplied by e^{-m}. The factor appears on top and bottom and cancels exactly. The result is not approximately the same; it is identically the same.

Which is why the trick is free. It is not a numerical approximation with a small error — it is an algebraic identity that happens to keep the intermediate values inside the representable range. After subtraction the largest exponent is always $e^0 = 1$, so overflow becomes structurally impossible.

Solution (d). Direct computation of e^{-1000} gives 0 — underflow this time rather than overflow. All three terms are zero, the sum is zero, and you get 0 ÷ 0, which is NaN again. Different failure, same outcome.

Subtract the maximum, which is −1000: the logits become 0, −1, −2, and you get the identical 0.665, 0.245, 0.090. So the same single fix handles both extremes, which is a good argument for it being the standard.

Common mistake. Believing the max-subtraction is an approximation that trades a little accuracy for stability. It is exact — part (c) is a proof, not an argument. Students who think it approximates will hesitate to use it in precision-sensitive code, which is precisely backwards.

12.2 Fitting a model onto the hardware you have

M · G

Problem. You have a 13-billion-parameter model and a single GPU with 24 GB of memory. (a) Weight memory in fp32, bf16, int8 and int4. (b) Which formats fit, allowing 30% overhead for activations and KV cache? (c) Your team proposes int4 to leave room for a longer context. What do you require before agreeing?

Solution (a). Bytes per parameter: fp32 is 4, bf16 is 2, int8 is 1, int4 is 0.5. Multiply by 13 billion.

fp32: 13 × 4 = **52 GB.** bf16: **26 GB.** int8: **13 GB.** int4: **6.5 GB.**

Solution (b). With 30% overhead, multiply each by 1.3 and compare against 24 GB.

fp32: 67.6 GB. No. bf16: 33.8 GB. No — and this is the painful one, because bf16 is the default and it misses by a third. int8: 16.9 GB. **Fits.** int4: 8.45 GB. **Fits comfortably**, with about 15 GB spare.

So int8 is the highest-quality format that fits, and int4 is the option that buys headroom.

Point out why the 30% matters. Without it, bf16 at 26 GB looks like a near-miss on a 24 GB card and someone will try it. With overhead accounted for, the gap is 10 GB and the answer is unambiguous. Weight memory alone is never the requirement — activations, the KV cache and workspace all sit alongside it, and the KV cache in particular grows with context length and batch size.

Solution (c). Three requirements, and the first is non-negotiable.

A quality measurement on your own tasks, before and after. Int4 has a real accuracy cost — not a rumoured one — and its size varies enormously by model and task. Extraction and classification often survive it well; multi-step reasoning frequently does not. Nobody can tell you the cost from theory; you measure it.

And that measurement must be adequately sized. Per problem 3.5, a 30-prompt comparison cannot resolve anything smaller than about 18 points. If int4 costs you 4 points of accuracy, you need several hundred prompts to see it at all — which means a small evaluation will report "no difference" and you will ship a real regression believing it is free.

Second, name what the headroom is for, in numbers. "A longer context" is not a specification. How long? Recall from problem 7.5 that the attention score matrix grows with the square of context length while activations grow linearly. Compute the actual memory at the target context before assuming 15 GB is enough — the quadratic term can consume it faster than expected.

Third, consider int8 with a shorter context as the alternative. The proposal frames this as int4-or-nothing, but the real choice is a trade between weight precision and context length, and both sides have measurable value. Test int8 at your current context against int4 at the longer one, on the same evaluation set, and let the numbers decide.

Common mistake. Comparing weight memory against total VRAM with no overhead allowance — concluding that a 26 GB model fits on a 24 GB card "with a bit of tuning". It does not, and the failure arrives as an out-of-memory crash under production load rather than in testing, because the KV cache grows with concurrent requests.

12.3 "I implemented it straight from the paper"

M

Problem. No arithmetic. An engineer has written their own softmax, log-probability and normalisation functions, transcribed directly from the formulas in a paper, because "the library version has too much indirection and I wanted to understand it". Tests pass on small examples. Explain the risk and give a policy.

Solution. Credit the motive first: implementing something to understand it is a genuinely good instinct, and this session has done exactly that for a dozen formulas. The problem is shipping it.

Why the tests passing is not reassuring. Numerical failures are triggered by magnitude, not by logic. A softmax over logits of 2, 1 and 0 works perfectly in every implementation. The same code over logits of 1000 returns NaN, and over −1000 also returns NaN. Small test cases exercise the arithmetic and never the range.

Which means the code will pass review, pass CI, run correctly in staging, and fail in production when some input produces unusually large logits — a very long prompt, an unusual token, a temperature near zero dividing the logits up. The failure is data-dependent and therefore intermittent, which is the most expensive kind to diagnose.

The deeper point, which is the lesson of this topic. The formula in the paper and the code in the library are mathematically identical and computationally different. The library contains the max-subtraction from 12.1, log-sum-exp for log-probabilities, epsilon guards against division by zero, and a dozen similar adjustments — none of which appear in the paper because they are not mathematics. They are the accumulated scar tissue of people who hit these failures already.

So the "indirection" the engineer wanted to remove is largely the fixes. Transcribing the formula does not simplify the library; it reverts it.

The policy, in three parts.

Implement it yourself to learn — genuinely, do this, it is how you come to understand what the library is doing. Then delete it, or keep it in a notebook clearly marked as a teaching artefact.

Use the library implementation in anything that ships. Not because it is more correct mathematically, but because it is correct across a range your tests do not cover.

And if you genuinely must write your own — because you need a fused operation, or a variant that does not exist — then test at the extremes deliberately: values around ±700 where e^x approaches float limits, values near zero, exact zeros, and duplicated maxima. Write those cases first. The bug you are guarding against will never appear in a test built from realistic-looking numbers.

Common mistake. Reading this as "never write numerical code". The lesson is narrower and more useful: mathematical correctness and numerical correctness are separate properties, and unit tests built from plausible numbers verify only the first. Students who take away "libraries are magic, do not look inside" have learned the wrong half — the point of 12.1 was that the fix is understandable in two lines.

TOPIC 13 · FIVE PROBLEMS

Evaluation

Four boxes and a threshold. Every metric is a ratio of two of the boxes, and every decision is about what the errors cost.

How many problems earn competence here: five. One on filling the confusion matrix from a word problem, which is the skill students lack. Two on precision and recall, because the pair only becomes intuitive when you have computed it twice on different cost structures. One on threshold movement, so they see the trade rather than being told about it. One judgement problem on accuracy, which is the metric they will be shown in every meeting.

13.1 Filling four boxes from a word problem

A

Problem. A churn model runs on 5,000 subscribers. 400 actually churned. The model flagged 600 as at-risk, and 280 of those did churn. (a) Fill in all four boxes. (b) Compute precision, recall and accuracy. (c) Which single number would you put in a monthly report, and what would you say alongside it?

Solution (a). Extract carefully — this is where errors start, not in the arithmetic.

True positives = 280. Flagged and did churn. Given directly.

False positives = 600 – 280 = 320. Total flagged minus correctly flagged. These subscribers were flagged and stayed.

False negatives = 400 – 280 = 120. Total real churners minus those we caught. These left without warning.

True negatives = 5,000 – 280 – 320 – 120 = 4,280. Everyone else.

Check the totals both ways: 280 + 320 = 600 flagged, correct. And 280 + 120 = 400 churned, correct. Two independent checks, both free.

Solution (b). Precision = 280 ÷ 600 = **0.467.** Of everything flagged, under half churned.

Recall = 280 ÷ 400 = **0.70.** Of everyone who churned, we caught seven in ten.

Accuracy = (280 + 4,280) ÷ 5,000 = 4,560 ÷ 5,000 = **0.912.**

Note the denominators, because that is what separates the two ratios. Precision divides by what the *system said* — 600. Recall divides by what was *actually there* — 400. Same numerator both times; only the denominator moves.

Solution (c). If forced to one number, report **recall at 0.70** — but the honest answer is that one number is the wrong request, and saying so is part of the job.

Why recall over precision here: the point of a churn model is to intervene before customers leave. A missed churner is revenue gone permanently. A false positive costs a retention offer sent to someone who was staying anyway — cheap, and occasionally even beneficial.

What to say alongside it: "we catch 70% of churn, and 53% of our outreach goes to people who would have stayed. At ₹X per retention offer, that costs ₹Y per month to save Z customers." That sentence turns two ratios into a business case, which is what a monthly report is for.

And explicitly do *not* lead with the 91.2% accuracy. Since only 8% of subscribers churned, a model that flagged nobody would score 92% — better than this one. Reporting accuracy would make a working model look worse than doing nothing, which is the opposite of informative.

Common mistake. Reading "flagged 600, of which 280 churned" as 600 false positives, or computing false negatives as 600 – 400. Have students draw the 2×2 grid physically and place each given number in a box before calculating anything. The extraction is the hard part; the division is trivial.

13.2 Same model, two businesses, opposite thresholds

A · M

Problem. A content-safety classifier flags policy-violating posts. On 10,000 posts, 200 genuinely violate policy. At threshold 0.5: TP 150, FP 900. At threshold 0.8: TP 90, FP 60. (a) Compute precision and recall at each. (b) Which threshold for a children's education platform? (c) Which for a general discussion forum where over-moderation drives users away? (d) State the principle.

Solution (a). Recall's denominator is 200 in both cases — the number of real violations does not change with your threshold.

At 0.5: flagged = 150 + 900 = 1,050. Precision = 150 ÷ 1,050 = **0.143.** Recall = 150 ÷ 200 = **0.75.**

At 0.8: flagged = 90 + 60 = 150. Precision = 90 ÷ 150 = **0.60.** Recall = 90 ÷ 200 = **0.45.**

The trade is stark. Raising the threshold quadrupled precision, from 0.14 to 0.60, and cost 30 points of recall. Missed violations went from 50 to 110.

Solution (b). Threshold 0.5, for the children's platform.

The costs are wildly asymmetric. A violating post reaching a child is a serious harm, potentially a regulatory matter, and unrecoverable once seen. A false positive means a harmless post is held for review and a moderator releases it twenty minutes later — a minor annoyance.

Accept precision of 0.143. Yes, six in seven flags will be innocent, and 1,050 posts per 10,000 need review. That is a staffing cost you can compute and pay. The alternative is 110 violations reaching children instead of 50, and that is not a cost you can pay.

In fact, for this platform you should ask whether an even lower threshold is warranted, and where recall stops improving.

Solution (c). Threshold 0.8, for the general forum.

Here the asymmetry runs the other way. Adult users who have posts wrongly removed leave and complain publicly — and at threshold 0.5 you would be wrongly flagging 900 posts per 10,000, which is 9% of all content. That is a product-destroying rate of over-moderation.

A missed violation on a general forum is bad but usually recoverable: users report it, a moderator removes it, and the damage is bounded. So accept recall of 0.45 and combine it with user reporting, which effectively raises your real recall without touching the classifier.

Solution (d). The principle: the threshold is not a property of the model, it is a statement about the relative cost of your two error types. Identical model, identical scores, opposite correct answers — because the two businesses lose different amounts on each mistake.

Which means a threshold cannot be chosen by a data scientist alone, and it cannot be chosen by maximising F1. It requires someone who knows what a missed violation and a wrongful removal each cost the business. If nobody has stated those two numbers, the threshold has been chosen by accident.

Common mistake. Computing F1 at both thresholds — 0.240 at 0.5, 0.514 at 0.8 — and selecting 0.8 for both businesses. F1 assumes precision and recall matter equally, which is exactly the assumption these two businesses violate in opposite directions. F1 is a convenience for reporting one number, not a decision rule.

13.3 Walking the threshold down a score table

M

Problem. A defect-detection model scores 1,000 manufactured parts. Real defects: 50. Scores band as follows — 0.9–1.0: 30 defective, 5 good. 0.7–0.9: 12 defective, 25 good. 0.5–0.7: 5 defective, 120 good. 0.3–0.5: 2 defective, 300 good. Below 0.3: 1 defective, 500 good. (a) Compute precision and recall at thresholds 0.9, 0.7, 0.5 and 0.3. (b) Where does the trade turn bad? (c) A missed defect costs ₹8,000 in warranty claims; inspecting a flagged part costs ₹150. Which threshold minimises total cost?

Solution (a). Accumulate from the top band down. Recall's denominator is 50 throughout.

Threshold 0.9: flagged 30 + 5 = 35. Precision 30 ÷ 35 = **0.857.** Recall 30 ÷ 50 = **0.60.**

Threshold 0.7: defective 30 + 12 = 42, good 5 + 25 = 30, flagged 72. Precision 42 ÷ 72 = **0.583.** Recall 42 ÷ 50 = **0.84.**

Threshold 0.5: defective 47, good 150, flagged 197. Precision 47 ÷ 197 = **0.239.** Recall 47 ÷ 50 = **0.94.**

Threshold 0.3: defective 49, good 450, flagged 499. Precision 49 ÷ 499 = **0.098.** Recall 49 ÷ 50 = **0.98.**

Solution (b). Look at what each step down buys and costs.

0.9 → 0.7: recall +24 points, precision −27. Reasonable — you gain 12 real defects at the cost of 25 extra inspections.

0.7 → 0.5: recall +10 points, precision −34. Getting worse. Five more defects for 120 more inspections.

0.5 → 0.3: recall +4 points, precision −14. Two more defects for 300 more inspections. Clearly bad.

So the trade turns bad after 0.7, and by 0.3 it is absurd. And notice this is readable from the raw bands before any arithmetic: the 0.3–0.5 band holds 2 defects against 300 good parts. Any threshold that admits that band is buying almost nothing.

Solution (c). Now put money on it. Total cost = missed defects × ₹8,000 + flagged parts × ₹150.

At 0.9: missed 20 → ₹160,000. Inspections 35 → ₹5,250. Total **₹165,250.**

At 0.7: missed 8 → ₹64,000. Inspections 72 → ₹10,800. Total **₹74,800.**

At 0.5: missed 3 → ₹24,000. Inspections 197 → ₹29,550. Total **₹53,550.**

At 0.3: missed 1 → ₹8,000. Inspections 499 → ₹74,850. Total **₹82,850.**

Minimum at threshold 0.5, at ₹53,550.

And this is the instructive part: 0.5 is where the precision-recall trade already looked poor — precision of 0.239 means three flags in four are false. Yet it is the cheapest option, because a warranty claim costs 53 times an inspection. The cost ratio overrides the metric intuition entirely.

So the lesson is not "0.7 is the right threshold because the trade turns there". It is that the metrics tell you the shape of the trade and the costs tell you where to sit on it. Change the warranty cost to ₹800 and the answer moves to 0.7. Same model, same bands, different business.

Common mistake. Choosing 0.7 from part (b) and never doing part (c). The precision-recall curve identifies the elbow; only the cost model identifies the optimum, and they frequently disagree. Any threshold decision without two cost numbers attached is a guess wearing a metric.

13.4 Evaluating a RAG assistant with no labels

G

Problem. Your document assistant answers free-text questions. There is no single correct answer to compare against, so precision and recall as defined above do not obviously apply. (a) Name three things you can still measure with a confusion matrix. (b) For retrieval specifically, define precision and recall. (c) You have budget to label 300 examples — how do you spend it?

Solution (a). Free-text output does not prevent binary evaluation; it just means you must choose *which* binary question to ask. Three that work.

Refusal correctness. For each question, should the system have answered or declined? Then a false positive is answering an unanswerable question — the hallucination case from 5.5. A false negative is refusing a question your documents could have answered. Both are countable, and this matrix is often the most valuable one in a RAG system.

Faithfulness. For each claim in each answer, is it supported by the retrieved documents? A false positive is an unsupported claim. This turns "did it hallucinate" into a countable rate rather than an anecdote.

Citation correctness. If the system cites sources, does each citation actually contain what it is cited for? Easy to label and a strong proxy for faithfulness.

Solution (b). For retrieval the definitions are clean, because relevance is a binary judgement per document.

Retrieval precision: of the chunks returned, what fraction were relevant. Low precision wastes context and feeds the model distractors.

Retrieval recall: of the chunks in the corpus that were relevant, what fraction did we return. Low recall means the answer was in your documents and the system never saw it — and note this failure is invisible downstream. The model cannot report missing what it was never given.

Recall is the harder one to measure, because it needs someone to know what the corpus contains. In practice you approximate it: for a set of questions, have a human find the ideal chunks by hand, then check how often retrieval surfaced them.

Solution (c). Do not spend all 300 on one thing, and do not spend them on end-to-end answer quality — which is the instinct, and it is the least diagnostic option. If a whole answer is wrong you still do not know whether retrieval or generation failed.

150 on retrieval relevance. Fifty questions, three retrieved chunks each, labelled relevant or not. Gives you retrieval precision directly, and per problem 5.4 also lets you set your threshold from data rather than intuition. This is the highest-value spend because retrieval failures cause generation failures.

100 on the refusal matrix. Fifty questions your documents can answer, fifty they cannot — and you must construct the second fifty deliberately, because production logs contain almost none. That asymmetry is exactly what problem 3.6 warned about.

50 on faithfulness of the answers the system did give. Smallest allocation, because if retrieval precision and refusal behaviour are both healthy, unfaithfulness is usually rare — and if they are not healthy, fix those first.

Common mistake. Concluding that free-text systems can only be judged subjectively, or by a model-as-judge with no human labels at all. Both give you a number with no error bars and no diagnosis. Decomposing into binary sub-questions is what makes a RAG system measurable — and each sub-question tells you which component to fix.

13.5 "99.2% accurate" on the slide

A · M · G

Problem. No arithmetic. A vendor presents a medical-imaging triage model: "99.2% accurate on 50,000 scans." The condition it screens for appears in about 0.6% of scans. What do you say, and what four things do you request?

Solution. What you say, first — and it should be said plainly, because it is checkable in the room.

A model that flags nothing at all would score 99.4% on this data. If 0.6% of scans show the condition, then 99.4% do not, and a system that always says "clear" is correct 99.4% of the time. The vendor's 99.2% is *below* that.

So the headline figure, taken literally, is consistent with a model that is slightly worse than a rubber stamp. It is almost certainly not that — the model probably does real work — but the metric chosen cannot distinguish between the two, which is why it should not be on the slide.

The mechanism, for the room: accuracy is dominated by true negatives. On a rare condition, the true-negative box holds 99% of all cases, and it is the one box nobody needed help with. Accuracy is mostly measuring how good the model is at the easy part.

The four requests.

One: the full confusion matrix, in counts. Not percentages — actual numbers of true positives, false positives, false negatives and true negatives. Every meaningful metric can be derived from those four, and no vendor can refuse without admitting something.

Two: precision and recall, with the threshold stated. For a triage model recall matters most — a missed positive scan is a missed diagnosis. And insist on the threshold, because precision and recall are meaningless without it; the same model produces any pair of values you like as you slide it.

Three: the prevalence in *their* evaluation set versus in your patient population. Per problem 2.3, identical models produce very different posterior confidence on different base rates. If they evaluated on an enriched set — deliberately oversampled positives, which is common — their precision will not survive contact with your 0.6%.

Four: how the 50,000 scans were sampled, and whether any were in training. Problem 3.6's lesson: 50,000 is a precise measurement of whatever population it came from. Three hospitals with one scanner model is not your hospital.

And the question that outranks all four: **what happens to a flagged scan, and what happens to a missed one?** If flagged scans go to a radiologist who was going to look anyway, high false positives are nearly free and you should push for maximum recall. If a "clear" result means nobody looks again, then every false negative is a patient sent home, and no accuracy figure in the world is the right thing to be discussing.

Common mistake. Winning the argument about accuracy and considering the evaluation complete. A competent vendor returns next week with precision, recall and F1 — all real numbers — measured on an enriched evaluation set at a threshold they chose. The base-rate critique defeats the *metric*; requests three and four are what defeat a misleading *measurement*.

TOPIC 14 · FIVE PROBLEMS

Agents & optimization

An agent is a scoring loop. Whatever you left out of the score is what the system will sacrifice.

How many problems earn competence here: five. One on utility as a weighted sum, one on argmax and what it returns, two on reward design — because reward design is where the real damage happens and one problem is not enough to feel it — and one judgement problem on a misbehaving agent. The mechanics are trivial; the traps are all in what the utility function omits, so the problems weight towards design rather than arithmetic.

14.1 Scoring four tools for one request

G

Problem. An agent must answer "what were our top five products by revenue last quarter?" Four tools, each scored on three factors — likely to have the data, speed, cost — with weights 0.6, 0.25, 0.15. Scores out of 10: SQL query [9, 7, 9]. Web search [2, 6, 8]. Internal doc search [5, 8, 9]. Ask a human [10, 1, 2]. (a) Compute each utility. (b) Which does argmax return? (c) "Ask a human" scored 10 on having the data — why did it lose? (d) What is missing from this utility function?

Solution (a). Each utility is a weighted sum — the same dot product from Topic 7, wearing a different hat.

SQL: $(0.6 \times 9) + (0.25 \times 7) + (0.15 \times 9) = 5.4 + 1.75 + 1.35 =$ **8.50.**

Web search: $(0.6 \times 2) + (0.25 \times 6) + (0.15 \times 8) = 1.2 + 1.5 + 1.2 =$ **3.90.**

Doc search: $(0.6 \times 5) + (0.25 \times 8) + (0.15 \times 9) = 3.0 + 2.0 + 1.35 =$ **6.35.**

Ask a human: $(0.6 \times 10) + (0.25 \times 1) + (0.15 \times 2) = 6.0 + 0.25 + 0.3 =$ **6.55.**

Solution (b). Argmax returns **SQL query** — and note it returns the *action*, not the score. Max would give you 8.50; argmax gives you "run the SQL query". You want the action.

Solution (c). Asking a human is the most reliable option available — a perfect 10 on having the data — and it still lost, because reliability is only 60% of the score. Its 1 on speed and 2 on cost dragged it to 6.55.

Trace the arithmetic: it won the reliability term by 0.6 points over SQL, and lost the speed term by 1.5 and the cost term by 1.05. Net loss of about 1.95.

That is the whole nature of a utility function. Every factor is negotiable against every other, at an exchange rate you set when you chose the weights. Nothing is a hard requirement unless you make it one — and if you needed "always defer to a human on financial figures" to be inviolable, a weighted sum is the wrong instrument. You need a constraint, not a weight.

Solution (d). Several things, and each omission is something the agent will happily trade away.

Risk of a confidently wrong answer. Web search scored 2 on having the data — but the score does not distinguish between "will fail to find anything" and "will find a plausible wrong figure from a competitor's report". Those have very different costs and identical utility.

Reversibility. A SQL read is harmless. If one option were "email the figures to the board", the cost of being wrong would be unrecoverable, and nothing in these three factors captures that.

Permissions. Does this agent have read access to the revenue tables? A utility function that scores an action the agent cannot legally perform will select it and then fail.

And the general point: **whatever you omit from the utility function, the agent is free to sacrifice.** It is not being reckless — it is optimising precisely what you gave it.

Common mistake. Treating the weights as objective. They encode a judgement — that having the data matters 2.4 times as much as speed, and 4 times as much as cost. Someone chose those ratios, probably quickly, and they now govern every decision the agent makes.

14.2 Cumulative return over an episode

M · G

Problem. A research agent earns +12 per correct retrieval, −6 per incorrect retrieval, −0.5 per step taken, and −30 for a fabricated citation. Episode A: 4 correct, 1 incorrect, 9 steps, no fabrication. Episode B: 6 correct, 0 incorrect, 22 steps, no fabrication. Episode C: 3 correct, 0 incorrect, 5 steps, 1 fabrication. (a) Compute each return. (b) Rank them. (c) Episode B found the most information — why did it not win? (d) What behaviour does this reward scheme encourage?

Solution (a). Return is the sum of rewards across the whole episode.

Episode A: (4 × 12) + (1 × −6) + (9 × −0.5) = 48 − 6 − 4.5 = **+37.5.**

Episode B: (6 × 12) + (22 × −0.5) = 72 − 11 = **+61.0.**

Episode C: (3 × 12) + (5 × −0.5) + (1 × −30) = 36 − 2.5 − 30 = **+3.5.**

Solution (b). B (61.0), then A (37.5), then C (3.5).

Solution (c). It did win — B has the highest return. The question is a deliberate trap, and it is worth letting students fall into it.

B took 22 steps, more than twice A's 9, which feels wasteful. But the step penalty is only −0.5, so 22 steps cost just 11 points while its two extra correct retrievals earned 24. The reward scheme, as written, is happy to pay 22 steps for 6 retrievals.

Is that what you intended? Perhaps not. If a step involves an API call costing real money and several seconds of user waiting, then −0.5 against +12 badly under-prices it. The agent will wander for twenty steps to find one more fact, because you told it that trade was worthwhile.

Set the step penalty to −2 and recompute: A becomes 48 − 6 − 18 = 24, B becomes 72 − 44 = 28. Still B, but the margin collapses from 23.5 points to 4. At −3, A wins. **The ranking of your agents' behaviour depends on a number somebody typed once.**

Solution (d). Three behaviours, readable directly from the ratios.

It encourages thorough searching — steps are cheap relative to retrievals, so exploring is rational.

It tolerates guessing. A correct retrieval is +12 and an incorrect one is −6, so an agent that retrieves recklessly and is right half the time nets +3 per attempt. The scheme pays it to guess. If you wanted caution, the penalty needs to exceed the reward.

And it treats fabrication as genuinely serious — at −30, one fabrication wipes out two and a half correct retrievals. Episode C did nothing else wrong and barely scored positive, which is the correct signal.

Common mistake. Judging episodes by counting good and bad events rather than computing the return. C looks respectable — 3 correct, 0 incorrect, efficient at 5 steps — until the fabrication term is applied. The return is the only thing the agent optimises, so it is the only thing you should rank by.

14.3 Designing rewards for a support agent

G

Problem. A support agent can resolve a ticket, escalate to a human, or close it. Management proposes: +10 for a resolved ticket, 0 for an escalation, +2 for a closed ticket. (a) What will this agent learn to do? (b) Trace the arithmetic that makes that rational. (c) Propose a corrected scheme and justify each number.

Solution (a). It will close tickets it cannot resolve, and it will almost never escalate.

Solution (b). Put yourself in the agent's position facing a difficult ticket it probably cannot solve — say a 30% chance of resolving it.

Attempt to resolve: expected value 0.3 × 10 = +3, and if it fails it presumably closes or escalates anyway.

Escalate: +0. Guaranteed nothing.

Close it: +2. Guaranteed, immediate, requires no work.

Closing beats escalating, always, for every ticket, by 2 points. So escalation is strictly dominated — there is no situation in which the agent should ever choose it. Management has written a rule that says "never ask for help", and they did it by leaving one number at zero.

And worse: closing a hard ticket at +2 competes respectably with attempting it. Below a 20% resolution chance, closing is the better bet. The agent will learn to triage by difficulty and quietly bin anything hard.

Note that nothing here is a malfunction. The agent is doing exactly what the numbers instruct. This is the sentence to repeat: **whatever you reward is what you get more of, and whatever you score at zero, you have forbidden.**

Solution (c). A corrected scheme, with reasoning for each number.

Resolved and confirmed satisfactory: +10. Keep it, but attach it to a confirmation signal rather than to the agent's own belief that it resolved something. Otherwise the agent is grading its own work.

Escalated appropriately: +6. The key change. Escalation is a *good outcome* — the customer gets helped by someone who can help. It should score most of what a resolution scores, because from the customer's point of view it nearly is one. Setting it to zero was the entire bug.

Closed without resolution: −8. Negative, and firmly so. An unresolved ticket closed is a customer abandoned, and it should never be the profitable option. The sign is what matters more than the magnitude.

Resolved incorrectly — customer returns within 48 hours: −12. Worse than escalating, worse than admitting defeat. This is what stops the agent from confidently guessing at hard tickets, which is the failure the original scheme actively encouraged.

Per-turn cost: −0.3. Small, to discourage endless clarifying questions, but far smaller than any outcome term so it never dominates.

Now check the difficult-ticket arithmetic again. Attempt: 0.3 × 10 + 0.7 × (−12) = 3 − 8.4 = −5.4. Escalate: +6. Escalation wins decisively, which is correct — a ticket the agent probably cannot solve should go to a human. Always re-run the marginal case after changing rewards; that is what tells you whether the scheme does what you intended.

Common mistake. Fixing this by raising the resolution reward to +20 to "motivate the agent to try harder". That widens the gap between attempting and escalating, making the agent *more* willing to guess at tickets beyond its competence. When an agent takes the wrong action, look first at what you scored at zero or made negative — not at whether the good outcome is rewarded enough.

14.4 A pricing agent that behaves badly

A · G

Problem. A ride-hailing pricing agent maximises expected revenue per ride. During a city-wide power failure it raised fares to 6× the normal rate. There is public outrage and a regulatory inquiry. (a) Did the agent malfunction? (b) Locate the failure precisely. (c) Propose two structurally different fixes and say which you would ship.

Solution (a). No. It maximised expected revenue per ride, which is what it was built to do. Demand spiked, supply was constrained, and the revenue-maximising price genuinely was very high. The optimisation worked perfectly.

Say that plainly to the class, because the instinct is to call it a bug. The mathematics is correct and the outcome is unacceptable, and holding both of those at once is the entire lesson of this topic.

Solution (b). The failure is in the utility function, and specifically in what it omits.

Revenue per ride was in the objective. Not in the objective: regulatory risk, brand damage, customer trust, the long-term value of customers who now delete the app, and any notion that an emergency is a different situation from a rainy Tuesday.

And per 14.1's principle — whatever you omit, the agent will sacrifice — it sacrificed all of them, efficiently, at scale, in minutes. A human pricing manager would have hesitated, not because they compute better, but because they carry an implicit utility function containing terms nobody wrote down.

That is the structural problem with automating a decision: you must make the implicit explicit, and you will forget something.

Solution (c). Fix one — add the missing terms to the utility function. Include a penalty proportional to how far price exceeds baseline, a term for expected churn, and a term for regulatory exposure.

Advantage: keeps a single coherent objective, so the agent can still trade sensibly at the margins. Disadvantage: you must put a number on brand damage, which nobody can do credibly — and if you set it too low the agent will still surge, just slightly less. You have also invited the same failure for the next thing you forgot.

Fix two — a hard constraint. Cap the multiplier at, say, 2.5× regardless of what the optimiser wants, and cap it lower still when an emergency flag is set. The agent optimises freely inside the permitted region and cannot leave it.

Advantage: it does not require pricing the unpriceable, and it cannot be traded away by a large enough revenue opportunity. Disadvantage: it is blunt, and it will occasionally forgo legitimate revenue.

I would ship the constraint, and add the utility terms afterwards as a refinement.

The reasoning: a weight is negotiable and a constraint is not. Any term you add to a weighted sum can be outvoted by a sufficiently large opportunity on the other side — which is exactly the scenario you are trying to make impossible. When an outcome must never happen, express it as a boundary the optimiser cannot cross, not as a cost it can decide to pay.

Common mistake. Answering "the agent needs better training data" or "it should have known there was an emergency". Both miss the level. Even with perfect knowledge of the emergency, a revenue-maximising objective would still surge — knowing about the power failure only sharpens its estimate of demand. The problem is the objective, not the information.

14.5 The agent that will not stop

M · G

Problem. No arithmetic. A research agent in production occasionally runs for hundreds of steps on a single request, making the same three tool calls repeatedly, until a timeout kills it. The bill for last month was four times forecast. Trace why this happens in terms of utility and reward, and give three fixes in priority order.

Solution. Work out what the loop looks like from inside the agent's decision rule.

At every step the agent scores its available actions and takes the argmax. Suppose it is stuck — the information it needs is not obtainable. The available actions are: call tool A, call tool B, call tool C, or give up.

Now, what utility does "give up" have? In most implementations, nobody assigned it one, or it is zero, or it is negative because failing to answer is bad. Meanwhile "call tool A" has some small positive expected utility — it might work this time.

So a small positive beats zero, and the agent calls the tool. The state has not meaningfully changed, so on the next step it faces the identical choice and makes the identical decision. Nothing in the loop degrades, so nothing breaks the tie. It will do this until an external timeout intervenes.

This is the same structural error as 14.3. There, escalation scored zero and was therefore never chosen. Here, stopping scores zero or negative and is therefore never chosen. In both cases the correct action was made unreachable by leaving its value unspecified.

And note there is no per-step cost in the agent's view. Its bill is not in its utility function. The forty-times-per-request API spend is invisible to the thing generating it.

Fix one, highest priority: a hard step limit. Twenty steps, then stop and report what you have. This is a constraint, not a weight, so it cannot be optimised around — and per 14.4, that is exactly what you want for an outcome that must never happen. Ship this today; it caps your financial exposure immediately.

Fix two: a per-step cost in the utility function, and a positive reward for stopping appropriately. Both halves are necessary. The step cost makes continuing progressively less attractive, so a long loop eventually becomes irrational rather than merely bounded. And "stop and report partial findings" must carry positive utility, or it remains dominated — an agent that is punished for admitting failure will never admit it.

Set these so that the marginal case works out: after several unproductive steps, stopping should win. Then verify by recomputing, as in 14.3.

Fix three: loop detection. Track recent tool calls and their arguments; if the agent repeats an identical call with no new information, penalise or block it. This addresses the specific pathology rather than the general one, which is why it is third — but it is the fix that stops the wasteful behaviour while still allowing long, genuinely productive runs.

Common mistake. Shipping only the step limit and considering it solved. The limit caps the cost but the agent still burns twenty steps on every hopeless request and still reports failure at the end. The utility fix is what makes it stop *early* when stopping is right — the limit is a safety net, not a strategy.

INSTRUCTOR REFERENCE

How many problems, and why

Sixty-eight problems across fourteen topics. The allocation is driven by the number of distinct traps a topic contains, not by how important the topic sounds — a topic with one idea and one trap does not earn six problems, however central it is to the field.

TOPIC	N	ROLES	WHY THAT NUMBER
01 Counting	5	A M G	Two traps: order or not, pool or lists. Five makes both reflexive.
02 Probability & Bayes	6	A M G	Five traps, each ruining a real decision invisibly. Eight would be defensible.
03 Distributions	6	A M G	Two halves — distribution choice and sampling error — and the second is used weekly.
04 Vectors	5	A M G	One mechanical skill, three conceptual traps. Arithmetic is learned by problem two.
05 Similarity	5	A M G	Two problems on the threshold, because it is the highest-leverage number in retrieval.
06 Graphs	3	A M G	Vocabulary plus one trap. More would be teaching graph algorithms, a different course.
07 Matrices	6	A M G	The one topic where mechanics genuinely need drilling. Six is the floor, not the ceiling.
08 Logs	4	A M G	A supporting topic: exactly four downstream uses, so exactly four problems.
09 Gradients	6	A M G	Two on the loop, three on diagnosis, one on cost. No calculus drills — it is automated.
10 Softmax	5	A M G	Temperature is counter-intuitive and gets two problems; the $\tau=0$ myth gets its own.
11 Information theory	4	M G	Four distinct uses: entropy, cross-entropy, perplexity, drift. No fifth exists here.
12 Numerical stability	3	M G	One idea, one trap. Padding it would teach that problem count signals importance.
13 Evaluation	5	A M G	Precision and recall need computing twice on opposite cost structures to become intuitive.
14 Agents	5	A M G	Mechanics are trivial; two problems on reward design, where the damage happens.

IF YOU HAVE LESS TIME THAN THE FULL SET

The three-hour session cannot carry sixty-eight problems. Two reduced paths, both tested against the trap list rather than assembled by taste.

A single ninety-minute clinic — fourteen problems, one per topic. Take 1.2, 2.2, 3.5, 4.2, 5.4, 6.1, 7.3, 8.1, 9.3, 10.2, 11.1, 12.1, 13.2, 14.3. Each is the load-bearing problem of its topic: the one whose absence leaves a trap unaddressed.

A three-hour workshop — twenty-eight problems, two per topic. Add 1.3, 2.3, 3.3, 4.5, 5.5, 6.2, 7.4, 8.2, 9.1, 10.5, 11.4, 12.2, 13.3, 14.5. The second problem in each pair is deliberately a different *kind* — where the first is mechanical the second is judgement, and vice versa.

What not to cut. Never drop the second probability problem; the base-rate trap does not survive a single exposure. Never drop the matrix-shape problem, 7.3, since it is the most frequently used skill in the document. And never cut all the judgement problems to save time — they are what stop the session being remembered as arithmetic.

THREADS THAT RUN ACROSS TOPICS

Several problems are deliberately linked. Point the connections out when you reach the second half of each pair — the recognition is worth more than either problem alone.

Base rates decide everything. 2.2 → 2.3 → 2.5 → 13.5. The same arithmetic in four costumes: fraud, segments, hallucination detection, medical imaging.

The dot product, four times. 5.1 similarity → 7.1 scoring → 7.4 attention → 14.1 utility. One operation, four purposes.

Missing information cannot be thresholded away. 6.3 fraud rings → 5.5 retrieval → 14.4 pricing. If the signal is not in the input, no threshold recovers it.

Sample size before comparison. 3.5 → 4.3 → 12.2. A 3-point gap on 200 queries is noise, and it is the same noise sizing an index or a quantisation decision.

Thresholds are business decisions. 5.4 → 13.2 → 13.3. Metrics show the shape of the trade; only costs identify the optimum.

Whatever scores zero is forbidden. 14.3 escalation → 14.5 stopping. Two agent failures, one cause: the right action was left unvalued.